

Mission-Invariant Architecture Morphing: Service-Graph Reconfiguration Against Post-Access Reconnaissance, with Cryptographic Epoch Isolation and Mission-Domain State Continuity

Thor Thor

Independent Open-Source Researcher, THOR-SEC

ORCID: 0009-0001-6573-385X codethor@gmail.com

GitHub: <https://github.com/codethor0> <https://codethor0.github.io/thor-sec/>

This research was conducted independently on the author’s own time and is not sponsored by, affiliated with, or representative of any employer.

Preprint, September 2026. Released openly for community review.

Abstract

Modern moving-target defenses alter addresses, ports, hosts, software variants, workflows, or computing environments to reduce the useful lifetime of attacker observations. This paper proposes Mission-Invariant Architecture Morphing (MIAM), a research architecture that moves the defensive transformation boundary into the application itself, targeting post-initial-access reconnaissance such as dependency mapping, lateral movement, and credential reuse. MIAM represents a workload as capability units assigned to runtime service graphs drawn from a certified grammar. For each candidate graph, machine-evaluable attack-prerequisite predicates estimate which previously learned conditions remain actionable, while a separate exposure term bounds new attack surface introduced by the candidate. A graph delta is combined with a capability-state map to derive a transition policy that permits only validated mission-domain state to cross the epoch boundary while blocking designated runtime and security state. A verified target graph is then instantiated under distinct epoch authority and cut over behind a stable logical interface. The paper develops an operational reconnaissance-transfer model, architecture-distance measures, a recurrence-aware retention model showing that a finite variant pool imposes a nonzero knowledge floor, a cryptographic epoch-isolation construction, a graph-delta-driven State Continuity Firewall, an enterprise implementation path, an ablation-based experimental protocol, and a reference claim set released as a defensive publication. The work is a design proposal produced through independent open-source cybersecurity research; it reports no empirical security advantage and makes no patentability determination.

Keywords: moving target defense; self-adaptive systems; microservices; cyber resilience; reconnaissance; runtime architecture graphs; workload identity; key rotation; mission-domain state; attack surface

1. Introduction

A large class of intrusions becomes easier after the adversary learns stable facts about a target: which components exist, how services depend on one another, which credentials traverse which trust edges, where valuable state resides, and which sequence of prerequisites leads from an exposed surface to a protected resource. Conventional hardening reduces exploitable weaknesses, while moving-target defense (MTD) attacks a different assumption: that information gathered during reconnaissance remains useful long enough to weaponize. NIST research has described MTD as selecting adaptations over system models while preserving operational integrity, and subsequent research has explored topology shuffling, service movement, software diversity, and workflow reconfiguration [1, 2, 3, 4].

The central hypothesis of MIAM is that network movement alone leaves a large amount of semantic reconnaissance intact. If an attacker learns that an Internet-facing gateway calls an authorization service, which calls a transformation service, which holds a privileged path to a data store, changing only IP addresses or container placement may preserve the attacker’s dependency map. MIAM therefore treats the application decomposition itself as a controllable security variable. The external mission remains stable while the internal service partition, dependency graph, implementation assignments, identities, and selected runtime attributes can change across bounded epochs. Because the external interface is deliberately invariant, MIAM’s primary leverage is against reconnaissance performed after initial access: internal dependency mapping, lateral movement, credential reuse, and persistence inside the application. It offers little against an attacker whose entire plan runs through the stable external contract.

This concept overlaps important prior work and prior patents. Architecture-based self-protection has already used runtime architectural models to select security adaptations [5]; service-oriented MTD has already optimized feasible service configurations; workflow MTD has already modeled frequent task reconfiguration; runtime software product lines already support safe reconfiguration; patents already cover dynamic network architectures, virtual-address mutation, stateful container hopping, dependency-aware device reconfiguration, layered information-asset churning, architecture swapping, and automated microservice splitting or merging. Accordingly, the contribution proposed here is not “a system that changes.” It is a narrower combined control loop that explicitly minimizes the transferability of attacker prerequisites across changes in service boundaries while bounding newly introduced exposure, verifying declared mission invariants, filtering state continuity, and invalidating epoch credentials and keys [6, 7, 8].

2. Research Question and Proposed Contributions

The research question is: Can a production distributed system preserve its externally required mission while deliberately making its internal architecture sufficiently nonstationary that reconnaissance knowledge decays faster than an attacker can reliably exploit it, without introducing new exposure or imposing unacceptable availability, consistency, observability, or operational cost?

1. A formal system model in which capability units can be repartitioned into different runtime service graphs across security epochs while a machine-evaluable mission contract constrains observable behavior.
2. Reconnaissance Transfer Risk (RTR), an aggregate metric whose per-prerequisite actionability values are produced by machine-evaluable predicates, attack-graph reachability, or reproducible replay evidence rather than unconstrained subjective scoring, paired with an absolute exposure term so that breaking old knowledge cannot be bought with new attack surface.
3. A recurrence-aware retention model that separates epoch-bound from structure-bound prerequisites and shows that selection from a finite certified variant pool leaves a nonzero long-run knowledge floor that deployments must measure and bound.
4. A State Continuity Firewall (SCF) whose transfer policy is compiled from the source-to-target graph delta and a capability-state map, so mission-domain state is reconstructed while designated runtime, security, executable, and stale authority state is denied by default.
5. Cryptographic epoch isolation using independently generated epoch master keys, graph-bound derivation context, scoped workload keys, short-lived workload identity, AEAD-protected state transfer, explicit cross-epoch authorization rules, and source-epoch revocation.

6. A constrained architecture compiler that chooses only among verified transformations or certified variation points, avoiding the intractability and safety risk of arbitrary runtime refactoring.
7. An ablation-based evaluation protocol that separates the contribution of graph morphing from the contribution of rejuvenation and credential rotation alone.
8. A reference claim set, released openly as a defensive publication, describing the integrated technical nexus among prerequisite-actionability evaluation, mission-constrained graph transformation, graph-delta-derived state policy, controlled cutover, and epoch authority invalidation.

3. Related Work and Prior-Art Landscape

3.1 Moving-target defense theory and dynamic security metrics

MTD is a mature research area rather than a new label for MIAM. NIST’s 2013 intelligent MTD design reasoned over network configuration, operational goals, and security goals to select adaptations while maintaining integrity [1]. Zhuang, DeLoach, and Ou subsequently proposed an initial formal theory of MTD, defining configuration state spaces and separating the problem of selecting the next configuration from the problems of adaptation selection and timing [2]. MIAM’s formal model in Section 5 is an application-level instantiation of that vocabulary: its configuration space is the certified service-graph family Ω_M , its adaptation-selection problem is Equation (23), and its timing problem is addressed in Sections 6.6 and 6.7. Yackoski, Li, DeLoach, and Ou described a mission-oriented MTD combining literal network modifications with semantic changes under cryptographically strong dynamics [9]; that work, and the self-shielding architecture it builds on, is the closest prior use of “mission” framing with cryptographically driven movement, and MIAM differs by moving the transformation into application service boundaries and state ownership rather than network attributes.

More recent work has moved critical services and data flows, optimized service configurations against attacker-defender models, and quantified the security/performance tradeoff of reconfiguration [8, 7]. Game-theoretic work already models the joint choice of which configuration to switch to and when, with switching cost depending on both the current and the next configuration [10]. Attack graphs are already used to drive MTD decisions: Yoon et al. shuffle SDN host configurations according to attack-path criticality [12], Zangeneh and Shajari select MTD moves with a cost-sensitive strategy over a Bayesian attack graph [13], and Liu et al. optimize microservice rotation against a microservice attack-graph model [14]. Evaluating candidate configurations against an attack graph is therefore not itself a contribution of MIAM. Dynamic MTD metrics already measure address variability, attack-path variability, and attack-stage success [11], and critiques of MTD have documented how movement can fail to be effective when the moved property is not the one the attacker depends on [15]. MIAM’s metric claims are therefore framed as an application-level extension rather than as the invention of dynamic security measurement.

A 2026 preprint formalizes configuration-pool indistinguishability and demonstrates that latency can fingerprint candidate MTD configurations; it also notes that an adversary denied one observable channel can pivot to others such as response size or CPU footprint [16]. Separately, MTDSense showed that an MTD-aware attacker can detect *when* an MTD has been triggered from traffic footprints and extract the movement interval to maximize the attack window [17]. A self-morphing architecture that emits a distinctive signature either for the active variant or for the transition event could reveal both to a capable observer. MIAM therefore includes variant leakage and transition detectability in its selection objective rather than assuming configuration diversity automatically creates uncertainty.

3.2 Cloud-native and microservice MTD

MTD has been applied directly to microservices. Torkura et al. proposed a cyber-risk-driven MTD for microservice architectures [18]. González Pineda et al. used Kubernetes load balancing to rotate among microservice versions implemented in different programming languages, which is prior art for MIAM’s implementation-substitution morph [19]. Ma et al. modeled lateral movement in cloud-native environments with attack graphs and formulated selective MTD as a constrained combinatorial optimization problem, evaluated on the TrainTicket benchmark [20]. MTD-Playground provides an attacker-aware evaluation framework with composite metrics for mutation-interval tradeoffs [21]. Mamaril et al. studied live migration of microservice payloads as an MTD action in Kubernetes [22], which makes state movement during microservice MTD established prior art. These works place, move, rotate, migrate, or diversify services; none that we reviewed repartitions capability-to-service boundaries while deriving the state-transfer policy from the resulting graph delta.

3.3 Software diversity and runtime software reconfiguration

Automated diversity predates MIAM. N-Variant Systems execute diversified variants on common inputs and detect divergent behavior, while dynamic software product-line research permits runtime movement among feature configurations under safety constraints [23, 6]. These works support a key design lesson: practical runtime diversity should be generated from controlled variation mechanisms with explicit correctness conditions, not from unrestricted source transformation in production.

3.4 Autonomic adaptation, self-protection, rejuvenation, and secure state continuity

MIAM also sits within a longer lineage of self-managing systems. Kephart and Chess described autonomic computing as systems that manage themselves against high-level objectives, a control-loop framing directly relevant to MIAM’s observe, estimate, propose, verify, transition, and learn cycle [25]. Yuan, Malek, Schmerl, Garlan, and Gennari applied architecture-based self-adaptation specifically to self-protection, using a runtime architectural model to select protective adaptations [5], and Schmerl, Cámara, Moreno, Garlan, and Mellinger applied the same architecture-based self-adaptation approach to MTD itself, reasoning about MTD tactics and their quality tradeoffs at the level of the running system’s architecture [24]. Together these are the closest conceptual antecedents to MIAM’s use of the application architecture as the defended and adapted object. The distinction proposed here is not the existence of an architecture-level feedback loop; it is the objective (continued actionability of previously learned attack prerequisites, bounded by absolute exposure) and the transition protocol (graph-delta-derived state policy and epoch authority isolation) applied to runtime service graphs.

The State Continuity Firewall also overlaps recovery research. Software rejuvenation formalized restarting software into a cleaner internal state, while microreboot work separated process recovery from data recovery so components could restart without discarding durable application state [26, 27]. Secure replication research likewise shows that state continuity, ordering, and authority have long-standing distributed-systems security constraints [28]. MIAM therefore treats clean reconstruction as prior-art context and narrows its contribution to deriving a state-transfer policy from the graph transition and mission-domain state contract. Because rejuvenation and credential rotation alone already remove many footholds, Section 15 requires an ablation that isolates the effect of graph morphing from those mechanisms.

3.5 Patent collision landscape

The broad anti-reconnaissance territory is crowded. US10447710B1 addresses self-shielding dynamic network architecture; US10958478B2 combines address mutation, container hopping, topology change, and state migration; and US11005886B2 changes network-level attributes while keeping applications substantially continuously available [29, 31, 32]. These references prevent MIAM from relying on generic movement, topology mutation, or continuity during network change as its inventive center. US8646085B2 already describes reconfiguring a technical system by applying candidate sets of countermeasures to a directed graph of potential attacks and ranking them by risk reduction [30], so security-graph-driven selection of a reconfiguration is also old.

At the graph and architecture level, US11729221B1 uses dependency information for reconfiguration; US11748490B2 composes moving-target layers and controlled churn, including key changes; US12124887B2 measures microservice relationships and can recommend service merging; and US12541603B2 swaps workspace architectures [33, 34, 35, 36]. EP4046020A1, CN121116381A, and CN120743522A further narrow the space by covering runtime decomposition or reconstruction of application services [37, 38, 39].

The remaining distinction is therefore intentionally narrow: a controller evaluates continued actionability of prior attack prerequisites against candidate application graphs, bounds the new exposure those candidates introduce, derives a state-transition policy from the actual source-to-target graph delta, rejects candidates that fail a machine-evaluable mission contract, establishes a separately authorized target epoch, and cuts over only after mission-domain state has crossed a validated state boundary. No reviewed reference is treated here as proof of novelty or non-obviousness.

This landscape is a technical prior-art screen, not a legal claim chart. Patent scope turns on the claims, prosecution history, jurisdiction, priority, and facts; Google Patents’ displayed legal status is not a legal conclusion. The combination risk is substantial, particularly across dependency-aware reconfiguration, moving-target churn, microservice restructuring, state migration, and continuous cutover. Organizations deploying MIAM should obtain their own freedom-to-operate analysis.

4. Threat Model and Design Goals

MIAM targets reconnaissance-dependent, multi-stage attacks against distributed applications, with primary emphasis on the post-initial-access phase. The attacker may observe public endpoints, protocol behavior, timing, errors, deployment artifacts, leaked configuration, and compromised low-privilege workloads. The attacker may attempt lateral movement, credential replay, dependency discovery, privilege escalation, exploitation of shared software weaknesses, persistence through mission-domain business objects, telemetry poisoning, active-architecture fingerprinting, detection of transition events, or exploitation of the source/target overlap window. The attacker is assumed to retain knowledge across epochs and may eventually learn every member of a finite certified variant pool. The attacker may also attempt to pin the system to a known graph by planting objects that cause transitions to abort, or by exhausting the candidate set through induced load, SLO pressure, or resource exhaustion until only variants the attacker has already mapped remain feasible. A compromised low-privilege workload may emit telemetry that is authenticated but false; MIAM assumes that estimates used for selection are corroborated by at least k sources in distinct trust domains, of which at most $f < k$ are adversarial, and treats estimates without that corroboration as unavailable. The root control plane is assumed uncompromised, but its reachable interfaces (mission-registry writes, grammar and variant supply chain, manifest signer, orchestrator API, attestation policy, and KMS authorization) are in scope as attack targets and are included in the exposure model of Section 6.3. The attacker is not assumed to break modern cryptography, compromise an isolated root control plane or

KMS/HSM at will, or instantaneously learn every target-epoch property.

The system is not a universal defense. Stable external APIs, business rules, data schemas, common dependencies, and other mission invariants can remain permanently learnable. A logic flaw exposed identically by every admissible graph can survive every morph. Supply-chain compromise across all variants, compromise of the mutation controller or verification authority, malicious-but-valid mission-domain state, shared persistent credentials, or destructive attacks that complete before an effective transition can defeat the approach. Conventional patching, least privilege, zero-trust identity, logging, backups, incident response, and control-plane hardening are prerequisites rather than substitutes.

Design goals are mission correctness, measurable reduction in prior-prerequisite actionability, no increase in absolute exposure beyond a declared bound, state hygiene, cross-epoch authority isolation, bounded transition overhead, low variant and transition fingerprintability, detection and containment of transition pinning, safe-point cutover, deterministic rollback, telemetry provenance, auditability, and vendor-neutral deployability. A cloud-native identity model is consistent with NIST’s shift from network location toward application and service identities in zero-trust architectures [40].

5. Formal System Model

5.1 Capability units, partitions, and service graphs

Let $C = \{c_1, \dots, c_n\}$ denote capability units. For a given certified architecture grammar, each capability unit is treated as an indivisible assignment unit with typed interfaces, declared data-access requirements, and security attributes. During epoch e , a partition $\Pi_e = \{S_{e,1}, \dots, S_{e,m}\}$ assigns each capability unit to exactly one deployed service unit. The service dependency graph is $G_e = (\Pi_e, E_e, A_e)$, where E_e is a directed set of permitted service-to-service dependencies and A_e contains implementation, placement, protocol, identity, and policy attributes.

$$G_e = (\Pi_e, E_e, A_e), \quad \bigcup_j S_{e,j} = C, \quad S_{e,i} \cap S_{e,j} = \emptyset \text{ for } i \neq j \quad (1)$$

In the terminology of [2], G_e is the active configuration and the certified family Ω_M defined below is the admissible configuration space.

A morph may fuse capabilities into a common service, split a service into certified components, substitute functionally equivalent implementations, insert a mediator or policy enforcement point, alter dependency routing, change placement, or rotate identity/protocol attributes. The first prototype should not synthesize arbitrary source code. It should select from a graph grammar or variant library whose modules and interfaces have already been tested.

5.2 Mission contract

A mission contract M specifies the externally required behavior and invariants that every target graph must satisfy: API schemas and allowed traces; authorization semantics; safety and liveness properties; consistency rules; data-residency constraints; resource limits; and service-level objectives (SLOs). Because general program equivalence is undecidable in the unrestricted case, MIAM operationalizes equivalence through constrained transformations, formal invariants where feasible, compatibility tests, state-machine checks, policy evaluation, and canary evidence.

$$G' \text{ is admissible iff } G' \models M \wedge \text{Policy}(G') = \text{true} \wedge \text{SLO}(G') \leq B \quad (2)$$

The set of admissible graphs reachable under the certified grammar is denoted Ω_M . For nondeterministic services, mission equivalence is expressed as an observational tolerance relation rather than byte-for-byte

output identity. If $\text{Obs}(G, x)$ is the externally visible trace for input x and d_M is a mission-specific distance, a candidate may require

$$d_M(\text{Obs}(G_e, x), \text{Obs}(G', x)) \leq \epsilon_M \quad \text{for certified } x \quad (3)$$

over a certified test domain plus invariant checks.

6. Quantifying Reconnaissance Transfer, Exposure, and Architectural Change

6.1 Reconnaissance Transfer Risk

Let $K_{\leq e} = \{\kappa_1, \dots, \kappa_k\}$ be the attack-relevant prerequisites or observations the attacker may hold at the end of epoch e . The set is cumulative: knowledge learned in any earlier epoch remains in $K_{\leq e}$ until the defender has evidence it is no longer actionable in any admissible graph. Each prerequisite κ_i carries an impact weight $w_i \geq 0$, a possession likelihood $c_i \in [0, 1]$, and a machine-evaluable actionability predicate P_i . The possession likelihood is the defender's estimate that the attacker holds κ_i , derived from detections, exposure duration, and observed access; it is not the attacker's internal confidence, which the defender cannot observe. The predicate encodes an attack precondition such as reachability of a target capability, continued acceptance of a credential, presence of a vulnerable implementation, existence of a required trust edge, or satisfaction of an exploit precondition. For candidate graph G' , actionability is

$$v_i(G') = \Pr [P_i(G', X) = 1 \mid E_i], \quad (4)$$

where E_i is the evidence available to the evaluator. A deterministic prototype may set v_i to 0 or 1 from graph/policy analysis; a replay-based prototype may estimate it as an observed success frequency with a confidence interval.

$$\text{RTR}(K_{\leq e}, G') = \frac{\sum_i w_i c_i v_i(G')}{\sum_i w_i c_i} \quad (5)$$

RTR lies in $[0, 1]$ when all weights and probabilities are in range. If $\sum_i w_i c_i = 0$, RTR is defined as 0 because the modeled attacker has no weighted prerequisite to transfer. RTR is not an established standard or theorem; it is a testable aggregation layer over operational prerequisite evaluators. Candidate actionability values must be traceable to predicates, attack-graph reachability, policy analysis, exploit-condition checks, or replay evidence, with the source of each estimate recorded in the evidence ledger [11, 41].

6.2 Attack-path survival

For attack path p containing prerequisite set K_p , path survival is most generally the joint probability that all required prerequisite events remain actionable in the candidate graph. The product form is valid only under the stated conditional-independence approximation. When prerequisites share credentials, hosts, trust edges, or exploit conditions, MIAM should use an attack graph, Bayesian network, factor graph, or direct replay estimate rather than multiply marginals.

$$S_p(G') = \Pr \left[\bigcap_{\kappa_i \in K_p} A_i(G') = 1 \mid E \right] \quad (6)$$

If the A_i are conditionally independent, $S_p(G') = \prod_{\kappa_i \in K_p} v_i(G')$.

$$\text{PSR}(G') = \frac{\sum_p W_p S_p(G')}{\sum_p W_p} \quad (7)$$

Path weights satisfy $W_p \geq 0$, and PSR is defined as 0 when $\sum_p W_p = 0$, so PSR lies in $[0, 1]$. RTR and PSR are correlated because they share prerequisite evaluations. They are reported separately because PSR captures path structure that RTR averages away, but their weights in Section 7 should be set with that overlap in mind.

6.3 Absolute exposure and privilege concentration

RTR and PSR measure how much of the attacker’s *existing* knowledge survives a morph. They say nothing about attack surface the candidate itself *introduces*: a new mediation service, a weaker implementation family, or a fused service that co-locates secrets. An optimizer that minimized RTR alone could therefore prefer a graph that breaks the attacker’s map while being easier to attack from scratch. MIAM adds an exposure term computed independently of $K_{\leq e}$.

Let I be the set of entry points (externally reachable interfaces, assumed-compromised footholds, and the reachable control-plane interfaces listed in Section 4), and let T be the set of protected assets with weights $\omega_t \geq 0$. For candidate G' , let $r_t(G')$ be the probability, or the 0/1 indicator in a deterministic prototype, that asset t is reachable from I in the attack graph generated for G' from its vulnerability, identity, and policy inventory [41]. Let $\sigma_c \geq 0$ be the sensitivity of the secrets and data access held by capability c . Then

$$\text{Reach}(G') = \frac{\sum_{t \in T} \omega_t r_t(G')}{\sum_{t \in T} \omega_t}, \quad \text{Conc}(G') = \max_{S \in \Pi'} \frac{\sum_{c \in S} \sigma_c}{\sum_{c \in C} \sigma_c}, \quad (8)$$

$$\text{Exp}(G') = \mu \text{Reach}(G') + (1 - \mu) \text{Conc}(G'), \quad \mu \in [0, 1]. \quad (9)$$

Conc captures the blast-radius side of service fusion: fusing capabilities can remove a network-visible trust edge while concentrating privileged material in one process. Each term is defined as 0 when its denominator is 0. An attack-surface metric such as that of Manadhata and Wing may be substituted for or combined with Reach [42]. Selection requires $\text{Exp}(G') \leq \text{Exp}(G_{\text{ref}}) + \delta_X$, where G_{ref} is a fixed, reviewed reference graph rather than the current graph. A fixed reference graph alone does not prevent ratcheting, because $\text{Exp}(G_{\text{ref}})$ itself moves if the scoring inputs change. MIAM therefore pins a versioned exposure-model specification X_v consisting of $I, T, \omega, \sigma, \mu$, the vulnerability and identity inventories, the probability calibration, and the attack-graph rules. $\text{Exp}(G_{\text{ref}})$ and $\text{Exp}(G')$ are always computed under the same X_v . When X_v changes, the reference graph and every candidate are rescored under the new version, and the rebaselining is signed and recorded in the evidence ledger. Under that discipline, exposure cannot ratchet upward through a sequence of individually small increases without an explicit, auditable rebaselining event.

6.4 Service-partition and graph distance

A system should not equate “different” with “secure.” Nevertheless, minimum structural diversity is useful as a constraint. For two partitions Π and Π' of the same n capability units, Variation of Information is $\text{VI} = H(\Pi) + H(\Pi') - 2I(\Pi; \Pi')$. For finite partitions of n labeled items, $\text{VI} \leq \log_2 n$, with equality attained between the all-singletons partition and the single-block partition; therefore $D_\Pi = \text{VI} / \log_2 n$ is bounded in $[0, 1]$ for $n \geq 2$ [43]. For $n \leq 1$ only one partition exists, and D_Π is defined as 0.

$$D_\Pi = \frac{H(\Pi) + H(\Pi') - 2I(\Pi; \Pi')}{\log_2 n} \quad (10)$$

A composite architecture distance can combine partition change, dependency-edge Jaccard distance, identity change, and implementation change. Coefficients are nonnegative and sum to one.

$$D_{\text{arch}} = \alpha D_\Pi + \beta \left[1 - \frac{|E \cap E'|}{|E \cup E'|} \right] + \gamma D_{\text{id}} + \delta D_{\text{impl}} \quad (11)$$

The edge term is defined as 0 when $E \cup E' = \emptyset$. Define $R_{\text{id}}(G)$ as the set of effective directed authorization relationships in graph G . Then $D_{\text{id}} = 1 - |R_{\text{id}}(G) \cap R_{\text{id}}(G')| / |R_{\text{id}}(G) \cup R_{\text{id}}(G')|$, with $D_{\text{id}} = 0$ when both sets are empty. Let $l_G(c)$ denote the implementation-family label for capability c and $\eta_c \geq 0$ its importance weight; $D_{\text{impl}} = \sum_c \eta_c \mathbf{1}[l_G(c) \neq l_{G'}(c)] / \sum_c \eta_c$, with $D_{\text{impl}} = 0$ if the denominator is zero. All terms therefore lie in $[0, 1]$.

6.5 Configuration entropy, variant leakage, and transition detectability

Let Ω_M be the set of architectures that satisfy M and all hard constraints. If candidate G is selected with probability $p(G)$, the selection entropy is

$$H(G) = - \sum_{G \in \Omega_M} p(G) \log_2 p(G). \quad (12)$$

High entropy alone is not enough. Externally observable latency, response size, errors, protocol quirks, or resource footprint can reveal the active variant. Let $\mathcal{E}$ be the hidden epoch/configuration identity and Y an observable feature vector spanning all channels available to the modeled attacker, not latency alone [16]. Normalized variant leakage is approximated by

$$F_{\text{obs}} = I(\mathcal{E}; Y) / H(\mathcal{E}), \quad (13)$$

defined as 0 when $H(\mathcal{E}) = 0$, with classifier success and ε -close configuration-pool tests as practical alternatives. For empirical evaluation, variant leakage can also be reported as cross-validated classifier advantage. For $m \geq 2$ equiprobable active graphs and classification accuracy Acc , define $F_{\text{cls}} = \max(0, (\text{Acc} - 1/m)/(1 - 1/m))$, with $F_{\text{cls}} = 0$ by convention when $m = 1$. $F_{\text{cls}} = 0$ corresponds to chance-level identification and $F_{\text{cls}} = 1$ to perfect graph identification; this is an estimator, not a cryptographic indistinguishability guarantee.

A second leakage channel is the transition event itself. Cold starts, cache misses, connection resets, certificate churn, and routing-weight changes can reveal *when* a morph occurred, which lets an attacker time its activity to the start of each epoch or infer the schedule [17]. Let Z be a binary label indicating whether an observation window contains a transition, and let AUC_Z be the cross-validated area under the ROC curve of the best available attacker-side detector. Transition detectability is

$$F_{\text{evt}} = \max(0, 2\text{AUC}_Z - 1), \quad (14)$$

which is 0 at chance and 1 for perfect detection. Mitigations include randomized intervals, pre-warming target graphs before cutover, gradual traffic shifting, and background churn that makes non-transition windows resemble transition windows.

6.6 Reconnaissance knowledge-retention half-time

The retention model formalizes the intuition that attacker knowledge should become perishable. The term used here is knowledge-retention half-time rather than physical half-life: it is a property of an assumed morph schedule and empirically estimated retention model, not an intrinsic property of reconnaissance.

Baseline geometric model. Suppose each deterministic morph occurs every τ seconds and leaves fraction ρ , $0 < \rho < 1$, of prior actionable knowledge. After n morphs, retention is ρ^n . A continuous interpolation gives

$$R(t) = \rho^{t/\tau} = \exp[(\ln \rho / \tau) t], \quad (15)$$

$$T_{50, \text{det}} = \tau \ln(1/2) / \ln \rho = \tau \ln 2 / \ln(1/\rho). \quad (16)$$

If morph events instead follow a Poisson process with rate λ and each event independently retains fraction ρ , $N(t) \sim \text{Poisson}(\lambda t)$. The probability-generating function gives $E[\rho^{N(t)}] = \exp[-\lambda(1 - \rho)t]$. In this

stochastic model, $T_{50,\text{Pois}}$ is the time at which expected retention reaches one-half; it is not the median time to a morph event.

$$E[R(t)] = \exp[-\lambda(1-\rho)t], \quad (17)$$

$$T_{50,\text{Pois}} = \ln 2 / [\lambda(1-\rho)]. \quad (18)$$

The geometric model is appropriate for *epoch-bound* prerequisites: epoch tokens, scoped keys, workload identities, and any other state that MIAM invalidates at every transition by construction. For those, ρ is ideally 0 per morph, independent of which graph is selected. At $\rho = 0$ the Poisson form still applies and gives $E[R(t)] = e^{-\lambda t}$ and $T_{50,\text{Pois}} = \ln 2 / \lambda$. Under a deterministic schedule with $\rho = 0$, the interpolation of Equation (15) does not apply: knowledge remains fully actionable until the first morph and drops to zero at $t = \tau$, so the morph period τ itself is the retention bound.

Recurrence in a finite variant pool. The geometric model is not appropriate for *structure-bound* prerequisites, whose actionability depends only on which graph is active: a trust edge, a co-location of capabilities, an implementation family. Because MIAM selects from a finite certified pool, graphs recur, and a structure-bound prerequisite that held in graph G_a becomes actionable again whenever G_a is re-selected. Let the selection policy induce a Markov chain on Ω_M with transition matrix Q and stationary distribution π , and let $A_i = \{G \in \Omega_M : v_i(G) = 1\}$ in the deterministic case. Starting from G_e , the expected actionability after n morphs is

$$E[v_i(G_{e+n})] = \sum_{G \in \Omega_M} Q^n(G_e, G) v_i(G) \xrightarrow{n \rightarrow \infty} \sum_{G \in \Omega_M} \pi(G) v_i(G) = \pi(A_i) \equiv R_{\infty,i}. \quad (19)$$

The long-run value $R_{\infty,i}$ is a floor, not zero. If Q is diagonalizable with eigenvalues $1 = \mu_1, \mu_2, \dots$, the approach to the floor is governed by the non-unit eigenvalues. Under Poisson morph arrivals with rate λ , the expected actionability is $e^{\lambda t(Q-I)} v_i$ evaluated at G_e , so

$$E[v_i(t)] = R_{\infty,i} + \sum_{k \geq 2} a_k e^{-\lambda(1-\mu_k)t}, \quad (20)$$

where the coefficients a_k depend on G_e and v_i . A retention half-time exists only when $R_{\infty,i} < 1/2$.

Worked case. Take $m = 4$ equiprobable certified graphs and a minimum-distance rule that forbids re-selecting the current graph, so $Q = (J - I)/3$ with J the all-ones matrix. The eigenvalues are 1 and $-1/3$, and π is uniform. A prerequisite valid only in the source graph has expected actionability $1/4 + \frac{3}{4}(-1/3)^n$ after n morphs: 0 after one morph, $1/3$ after two, and a floor of $1/4$. A prerequisite valid in two of the four graphs has expected actionability $1/2 + \frac{1}{2}(-1/3)^n$ and a floor of $1/2$, so its retention half-time never exists. Figure 1 compares these curves with the baseline geometric model at $\rho = 0.25$. The design consequences are direct: structure-bound prerequisites must be evaluated against the whole pool rather than against the next graph only, pools must be large and structurally diverse enough that $\pi(A_i)$ is small for high-weight prerequisites, and an attacker who has mapped every graph in a small pool gains little from morphing at all.

These equations are models, not guarantees. They become useful only when ρ , Q , and v_i are empirically estimated for a defined attacker and morph family. Correlated morph effects, persistent invariants, adaptive attackers, and multi-epoch knowledge accumulation violate the simplest models and must be measured rather than assumed away.

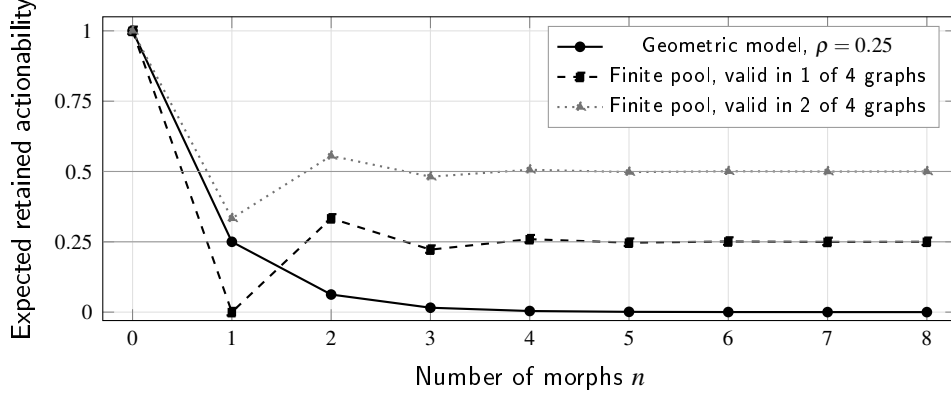


Figure 1: Retention of a structure-bound prerequisite under recurrence in a four-graph certified pool with no self-transition, compared with the geometric model. The finite pool converges to a floor equal to the stationary probability of graphs in which the prerequisite holds, not to zero.

6.7 Attacker completion-time model

A second model links morph rate and attacker speed. For an arbitrary nonnegative attacker completion-time random variable T_A and effective reconnaissance-decay rate a , expected retained reconnaissance at attacker completion is the Laplace transform $\mathcal{L}_{T_A}(a) = E[\exp(-aT_A)]$. If T_A is exponentially distributed with rate b and morphs are Poisson with $a = \lambda(1 - \rho)$, the expression reduces to

$$E[\exp(-aT_A)] = \frac{b}{b+a} = \frac{b}{b+\lambda(1-\rho)}. \quad (21)$$

Equation (21) applies directly to epoch-bound prerequisites. For structure-bound prerequisites, Equation (20) gives $E[v_i(T_A)] = R_{\infty,i} + \sum_{k \geq 2} a_k \mathcal{L}_{T_A}(\lambda(1 - \mu_k))$, so as the morph rate grows, expected retention at attacker completion approaches the floor rather than zero. The exponential case is an illustration, not an empirical claim about attacker timing; adaptive attackers may couple T_A to observed morph behavior, which is why F_{evt} matters.

This expression captures the strategic interaction: faster morphing helps, but so does making each morph semantically invalidate more prerequisites, and so does keeping the recurrence floor low. A high churn rate that preserves ρ near one, or that cycles among a few similar graphs, can be less useful than a slower morph that sharply reduces prerequisite transfer. This is precisely why MIAM optimizes transferability and pool composition, not frequency alone.

7. Morph Selection as a Constrained Optimization Problem

For each candidate G' , the controller evaluates prerequisite actionability, attack-path survival, absolute exposure, mutation cost, SLO risk, variant leakage, transition detectability, and state-continuity risk. It also computes a graph delta $\Delta_e(G') = \text{Diff}(G_e, G')$ that records capability reassignment, dependency-edge changes, implementation changes, identity-policy changes, and affected state owners. A candidate is eligible only if the mission contract, exposure bound, and transition-safety constraints pass; the optimizer then chooses among eligible candidates rather than allowing a risk score to waive correctness.

Each objective term is mapped to $[0, 1]$ before weighting: RTR, PSR, Exp, F_{obs} , and F_{evt} already lie in that range, while mutation cost C_{mut} , SLO risk L_{SLO} , and state risk S_{state} are divided by declared operator budgets and clipped, written $\hat{C}_{\text{mut}}$, $\hat{L}_{\text{SLO}}$, and $\hat{S}_{\text{state}}$. The raw quantities are defined as follows, each measured or predicted for one transition from G_e to G' :

- C_{mut} : additional compute consumed by the transition, in CPU-core-seconds summed over dark instantiation, pre-warming, state transfer, and overlap, plus control-plane and KMS operations weighted by a declared per-operation cost; budget B_C per transition.
- L_{SLO} : the fraction of the service's SLO error budget for the current rolling window that the transition is predicted to consume, estimated from staging load tests and canary history; budget B_L .
- S_{state} : the sensitivity-weighted fraction of registered mission-domain objects that must cross the SCF under P_{state} , plus a fixed penalty for each crossing object class that has no target-specific validator; budget B_S .

Each normalized term is $\hat{x} = \min(1, x/B_x)$ with $B_x > 0$ declared before operation.

$$J(G') = \lambda_R \text{RTR} + \lambda_P \text{PSR} + \lambda_X \text{Exp} + \lambda_C \hat{C}_{\text{mut}} + \lambda_L \hat{L}_{\text{SLO}} + \lambda_F F_{\text{obs}} + \lambda_V F_{\text{evt}} + \lambda_S \hat{S}_{\text{state}} \quad (22)$$

$$G_{e+1} = \arg \min_{G' \in \Omega_M} J(G')$$

subject to G' admissible under Eq. (2), $\text{VerifyStatePolicy}(P_{\text{state}}(G')) = \text{PASS}$,

$$\text{RTR}(K_{\leq e}, G') \leq 1 - r_{\min}, \quad \text{Exp}_{X_v}(G') \leq \text{Exp}_{X_v}(G_{\text{ref}}) + \delta_X, \quad D_{\text{arch}}(G_e, G') \geq d_{\min}. \quad (23)$$

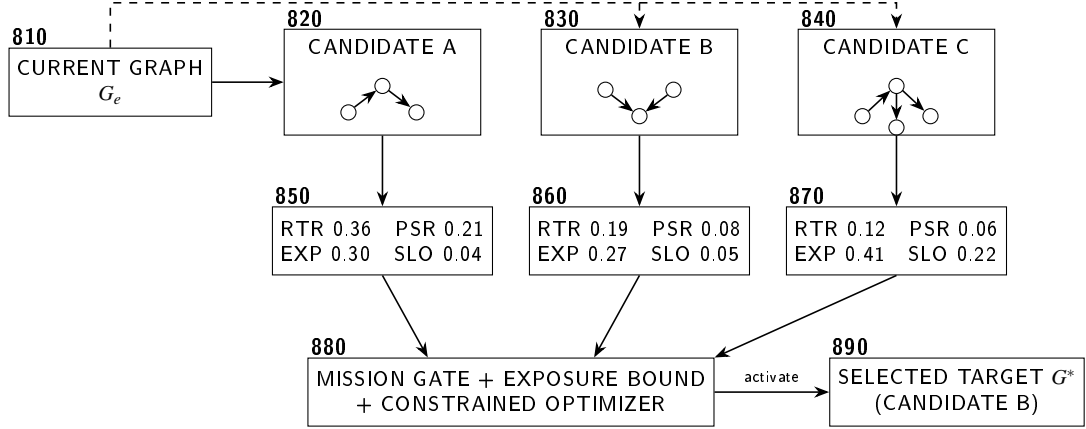
Here $P_{\text{state}}(G') = \text{CompileStatePolicy}(\text{Diff}(G_e, G'), M, \{\Sigma_c\})$ as defined in Section 11. If no candidate satisfies the constraints, the controller keeps the current graph or restores a known-good graph rather than relaxing a constraint. If the number of feasible candidates falls below a declared minimum $m_{\min}$ for longer than a declared interval, the controller raises an alert, because a shrinking feasible set can indicate that an attacker is exhausting candidates through load, SLO pressure, or resource starvation.

Equation (23) constrains each individual transition. Two further constraints apply at the level of the selection policy and schedule rather than to a single candidate, and are checked whenever the certified pool, the selection policy, or the morph schedule changes: for epoch-bound prerequisites, the knowledge-retention half-time of Equations (16) or (18) must not exceed a policy maximum $T_{\max}$; and for each high-weight structure-bound prerequisite κ_i , the recurrence floor $R_{\infty, i} = \pi(A_i)$ of Equation (19), computed from the stationary distribution of the selection policy, must not exceed a policy maximum $R_{\max}$. A pool, policy, or schedule that fails either check is rejected before any transition is attempted. The weights are not security constants. They are operator policy parameters that must be fixed or tuned against measured objectives. Operators that prefer not to scalarize may instead compute the Pareto front over the normalized terms and choose by lexicographic priority, for example exposure first, then RTR, then cost. In high-assurance environments, the candidate generator is a proposal engine only: the verification authority signs an approved transition manifest, and the orchestrator and key authority accept only a manifest whose graph digest, mission-contract version, state-transition policy, and authorization policy have been validated.

8. Why the Architecture Must Be Constrained

Arbitrary repartitioning grows super-exponentially. The number of partitions of n capability units is the Bell number B_n : $B_5 = 52$, $B_8 = 4,140$, $B_{10} = 115,975$, $B_{15} = 1,382,958,545$, and $B_{20} = 51,724,158,235,372$. Exhaustive search over all partitions is therefore unreasonable even before dependency edges, implementation variants, placement, identity, and policy attributes are considered.

The production design uses a certified architecture grammar rather than the full Bell space. If q independent variation points expose at most b_j legal alternatives, the unpruned candidate bound is $N_{\Gamma} \leq \prod_{j=1}^q b_j$; dependency, policy, residency, state-ownership, exposure, and mission constraints prune this set before scoring. An exhaustive bounded selector therefore scales with the number of grammar candidates actually emitted, not B_n , while CP-SAT/SMT, branch-and-bound, or graph search can prune earlier. The implementation



Example bounds: $RTR \leq 0.30$ (A fails), $EXP \leq 0.35$ (C fails). C has the lowest RTR but is rejected for new exposure; B is the lowest-risk candidate that satisfies every constraint.

Figure 2: Candidate generation, multi-objective scoring, and constrained selection. Reference numerals are reused in Appendix A.

goal is not maximum morph diversity. It is the best verified transition available within operational cost and a bounded decision deadline. Section 6.6 adds a countervailing requirement: the certified pool must still be large and diverse enough that the recurrence floor for high-weight structure-bound prerequisites stays below the operator’s tolerance.

9. MIAM System Architecture

The MIAM system (100) comprises a control and decision plane and a runtime and transition plane. The control plane contains the Mission Contract Registry (120), Observability Graph Builder (130), attacker-knowledge model (140), candidate graph generator (150), prerequisite-actionability, exposure, and fingerprint scorer (160), verification authority (170), epoch orchestrator (180), and identity/key authority (190). The runtime and transition plane contains the stable service interface (110), the State Continuity Firewall (200), the source and target runtime graphs (210, 220), and the evidence ledger (230), which the control plane writes and auditors read. The control plane should be smaller, more static, separately authenticated, and more strongly isolated than the workloads it controls because compromise of the mutation authority would invert the defensive advantage.

A concrete trust split is required. The candidate generator has no activation privilege; the verification authority signs an immutable transition manifest; the orchestrator activates only signed manifests; and the KMS/HSM releases target-epoch material only for an authorized manifest digest and attested orchestrator identity. Telemetry used for graph and prerequisite estimation should be cross-checked across independent sources and carry provenance. Morph triggers are rate-limited and use hysteresis so an attacker cannot cheaply force repeated transitions. On control-plane uncertainty, the fail-safe action is to retain or restore a known-good graph rather than explore a new one.

The lifecycle is observe, generate, score, verify, prepare, transfer and rehydrate, cut over, revoke/retire, and learn (Figure 4). Morph triggers may be periodic, risk-adaptive, operator initiated, incident driven, or chosen by an optimization policy. Purely deterministic schedules are easier to operate but more predictable and easier to detect; purely random schedules can waste resources. The preferred design combines bounded randomization with risk and mission constraints.

100 MIAM SYSTEM

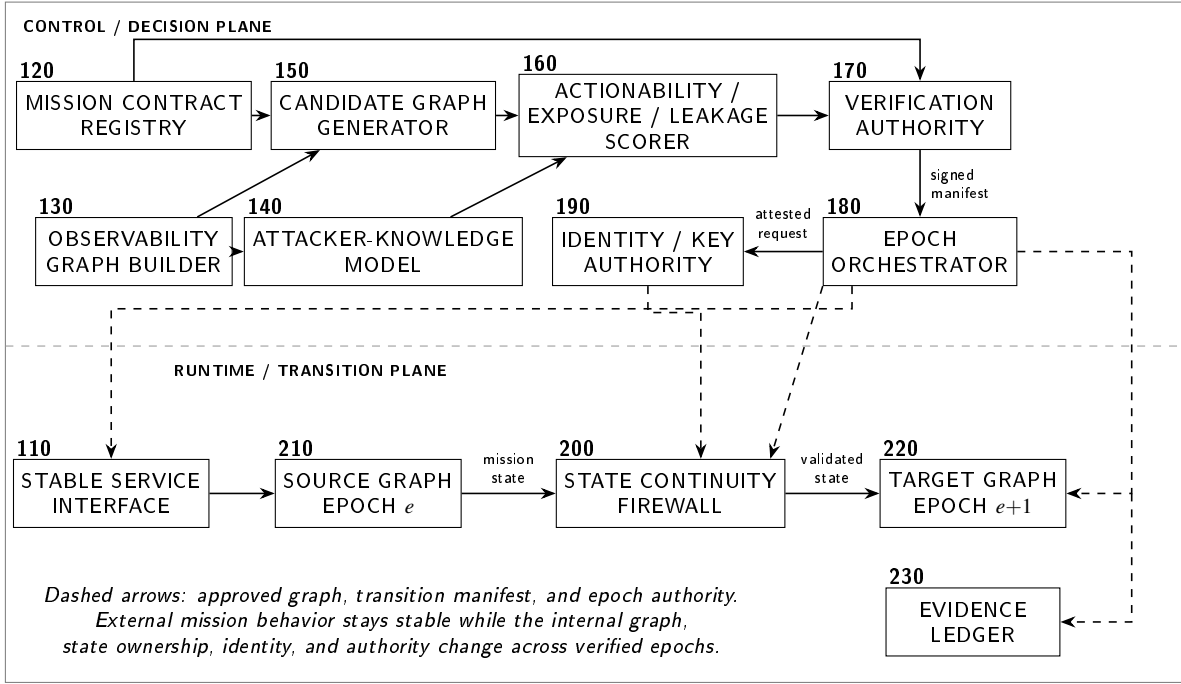
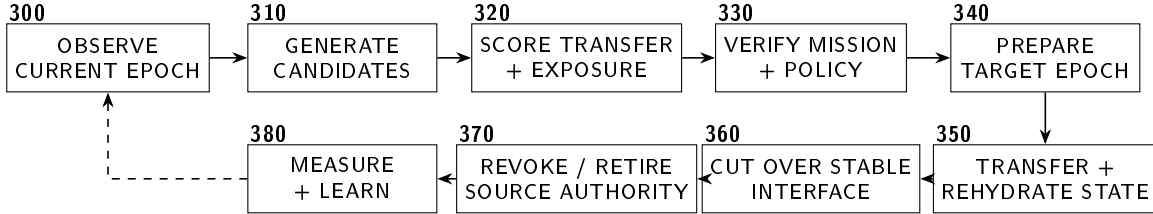


Figure 3: Reference architecture for mission-invariant service-graph morphing.



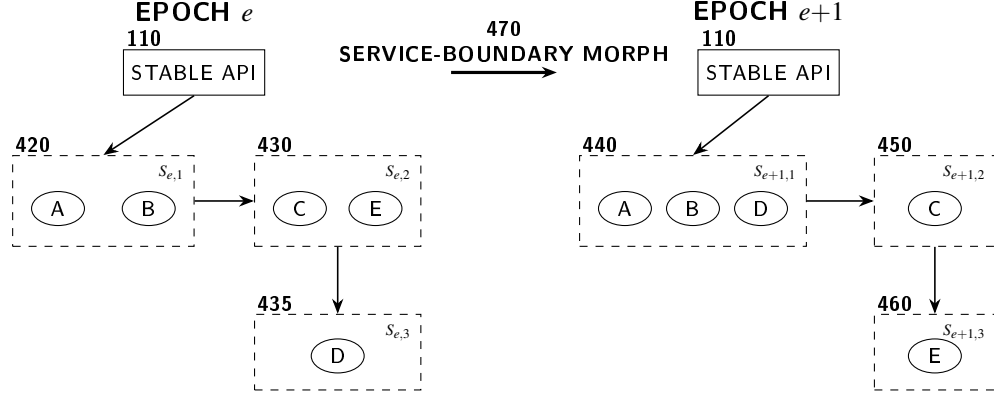
A candidate rejected at 320 or 330 returns control to 300 with the current or a known-good graph retained. Only a mission-compliant, exposure-bounded target proceeds through state transfer, cutover, and source-authority retirement.

Figure 4: Epoch transition lifecycle with closed-loop evidence feedback.

10. Service-Graph Morphing Mechanisms

Each morph changes one or more attack-relevant internal properties while preserving the mission contract. Service fusion can remove a network-visible trust edge but raises privilege concentration; service fission can create a new mediation boundary but may add edges; implementation substitution can change a vulnerability family; policy-mediator insertion can force a new authorization path; dependency rerouting can alter traversal; and placement or substrate changes can reduce reuse of host-specific knowledge. The system records the source-to-target graph delta and prefers transitions whose measured or predicted prerequisite actionability is materially lower without raising absolute exposure, not transitions that are merely structurally different. Figure 5 shows a service-boundary morph.

Runtime microservice decomposition and service reconstruction are themselves known in the patent literature, including runtime decomposition for distributed execution and recent dependency-graph-driven reconstruction techniques [38, 37, 39], and runtime rotation among implementation variants has been demonstrated on Kubernetes [19]. Therefore, MIAM's claim-oriented distinction is the security selection loop and



Capability units A to E (411 to 415) are repartitioned into a different service graph while the stable API and mission contract are preserved. $D_{\Pi} = 0.410$ for this morph (Section 16).

Figure 5: Example change in capability-to-service partition while the stable service interface and mission capabilities remain unchanged.

epoch lifecycle, not the general ability to split, merge, or diversify services.

11. State Continuity Firewall

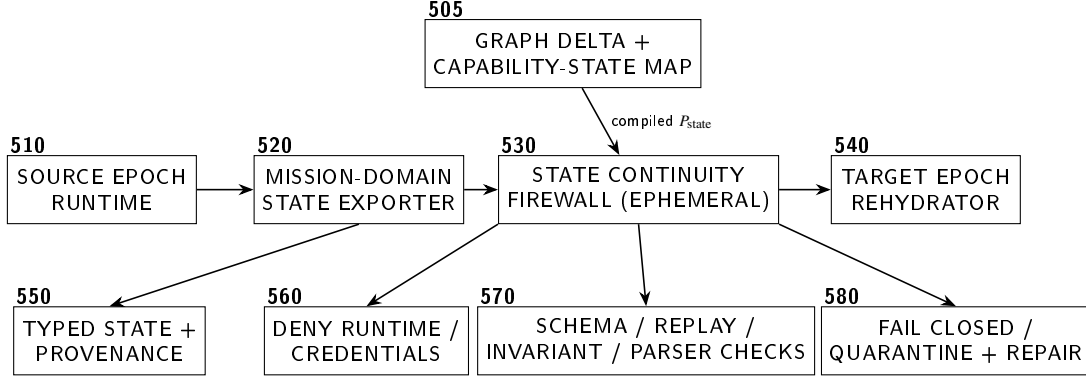
State continuity is where a moving architecture can accidentally preserve an attacker foothold. A conventional stateful migration can carry process memory, compromised caches, runtime tokens, injected libraries, or contaminated temporary artifacts into the target. MIAM therefore treats state movement as a derived transition policy rather than a generic copy operation. Mission-domain state is defined as typed state explicitly required by the mission contract or a registered capability-state contract to continue externally required behavior; being syntactically valid business data is not sufficient.

For each capability c , a capability-state contract $\Sigma_c = (T_c, V_c, R_c, X_c)$ declares transferable mission-domain object types T_c , required validators V_c , target reconstruction rules R_c , and prohibited runtime/security classes X_c . Given $\Delta_e = \text{Diff}(G_e, G_{e+1})$, the transition planner compiles $P_{\text{state}} = \text{CompileStatePolicy}(\Delta_e, M, \{\Sigma_c\})$. State classification is therefore tied to the actual capability ownership and boundary changes of the selected morph rather than to a global label such as “semantic.”

A state object may cross the SCF only if its type is allowed for the mapped target capability; its schema, provenance, integrity, authorization, freshness, and replay sequence validate; all mission/business invariants for that object hold; content-safety checks relevant to the target parser or deserializer pass; and the object is not a prohibited executable, credential, source-epoch token, process snapshot, hook, or local runtime secret. Durable mission records may pass after these gates. User-continuity state is reauthorized or rebound. Caches and derived indexes are rebuilt. Locks are reacquired under new fencing epochs. Source cryptographic material never crosses.

This stronger rule addresses malicious-but-valid business state. A payload can satisfy a schema yet still exploit a target parser or encode a dangerous workflow value. The SCF therefore applies target-specific semantic invariants, canonicalization, bounded parsing, content validation, and where applicable taint or provenance policy before rehydration. Failure is fail-closed: the object is quarantined and the target epoch remains uncommitted, or the transition is aborted if the object is mission-required. No partial interface cutover occurs merely to preserve availability.

The transfer manifest includes source and target epochs, graph digests, the graph delta, source-to-target capability mapping, object type, schema version, object identifier, monotonic sequence or nonce, prove-



Source-to-target transfer is permitted only through the validated state boundary during an epoch transition.

Figure 6: *State Continuity Firewall: graph-delta-derived transfer policy, mission-domain state validation, runtime-state denial, and target reconstruction.*

nance, authorization decision, validation policy version, and integrity/authentication evidence. During the overlap window, direct source-to-target workload calls are denied; the SCF transfer channel is the only authorized cross-epoch data path. A raw VM or container snapshot is never equivalent to mission-domain state.

Hardening the SCF. The SCF parses attacker-influenceable data exported from a possibly compromised source while holding transfer keys for both epochs, which makes it the highest-value component in the transition path. It should be instantiated fresh for each transition from a signed image, hold no persistent state beyond the transition, use memory-safe and bounded parsers, run with no network path other than the source export and target import channels, and be destroyed after the transition commits or aborts. A trusted execution boundary is an optional additional layer.

Pinning detection and containment. Fail-closed abort on mission-required objects creates a denial-of-morph risk: an attacker who plants one malformed but mission-required record could block every future transition and pin the system to a graph the attacker has already mapped. The SCF therefore routes a failed mission-required object to a quarantine-and-repair path (operator review, reconstruction from an authoritative system of record, or reissue by the owning business process) with a bounded retry budget. Repeated aborts attributable to the same object class or origin are raised as a security incident, and the orchestrator records the pinned duration in the evidence ledger so that extended pinning is visible rather than silent. When the pinned duration exceeds a declared threshold, the orchestrator applies containment to the pinned graph rather than waiting for a successful morph: it rotates epoch keys and workload identities in place, which invalidates epoch-bound prerequisites without any state crossing the SCF, and it isolates the capability that owns the failing object class behind a policy-enforcement point. These controls provide detection and containment, not a liveness guarantee. An attacker who can continuously inject distinct mission-required failures, or make the authoritative repair source unavailable, can still prevent service-graph morphing indefinitely. Forward progress for structure-bound prerequisites depends on the stated assumption that at least one authoritative reconstruction source for each mission-required object class remains available and uncompromised.

Overlap window and source writes. A compromised source may remain active during overlap and attempt to race writes, destructive actions, or tampering with the export. MIAM therefore revokes source

write authority at the transition fence: after the fence, the source has read-only access to the fenced snapshot that the exporter reads, and all mission writes are either rejected or queued for the target owner. The export is taken only from the fenced snapshot, and the snapshot digest is recorded in the transition manifest. The overlap window has a declared maximum duration T_{ov} ; if cutover has not completed by then, the orchestrator aborts and restores rather than extending the overlap. Destructive actions a compromised source completes before the fence are outside what a transition can undo and are handled by conventional backup and incident response.

12. Cryptographic Epoch Isolation

12.1 Key hierarchy and graph binding

Cryptography should support the morph rather than become the novelty by itself. For each epoch e , the key authority generates an independent fresh 256-bit epoch master key K_e inside a CSPRNG-backed KMS or HSM. The target key is not derived from the source key. This is important: compromise of K_e should not provide the derivation chain for K_{e+1} . NIST key-management guidance emphasizes lifecycle controls for cryptographic material; MIAM adds epoch scoping to that operational lifecycle [44, 45].

$$K_e \xleftarrow{\$} \{0, 1\}^{256} \quad \text{independently for each epoch } e \quad (24)$$

The controller canonicalizes the approved graph, mission-contract version, and policy version and hashes them to produce a graph context h_e . Canonicalization must be deterministic and unambiguous: the graph is serialized with a canonical encoding such as RFC 8785 JSON canonicalization [46] or the core deterministic encoding of CBOR defined in RFC 8949 [47]. Every multi-field input below is written $\langle \cdot \rangle$ and denotes a tuple encoding in which each field carries a fixed-width type tag and a fixed-width 32-bit big-endian length followed by its bytes, with the number of fields fixed per context label. Raw concatenation is never used, so no two distinct tuples share an encoding. SHA-256 is an example secure hash standardized by NIST; the architecture should remain algorithm-agile [48].

$$h_e = \text{SHA-256}(\langle \text{Canon}(G_e), M_{\text{version}}, \text{Policy}_{\text{version}} \rangle) \quad (25)$$

Because K_e is already a uniformly random key, no extraction step is needed; a single expansion step suffices. Scoped keys are derived with a PRF-based key derivation function keyed by K_e , for example HKDF-Expand with SHA-256 [49] or, in FIPS-validated deployments, a KDF in counter or feedback mode from NIST SP 800-108 keyed by K_e as a key-derivation key [50, 45]. The SP 800-56C two-step procedure is specified for shared secrets produced by key establishment and is not the relevant path here [51].

$$K_{e,s,p,j} = \text{KDF}(K_e, \langle \text{"MIAM-v1"}, \text{epoch_uuid}_e, \text{service}_s, \text{purpose}_p, \text{inst}_j, h_e \rangle, 256) \quad (26)$$

For purposes other than state transfer, inst_j is a fixed empty field. For the state-transfer purpose, inst_j is a 128-bit identifier generated by the KMS/HSM from its CSPRNG when an SCF instance is created, bound into the signed transition manifest, and recorded in the evidence ledger. The KMS/HSM refuses to perform a transfer-purpose derivation for any inst_j it has already served. A restarted or replicated SCF therefore always receives a new identifier and a new key, so an instance-local nonce counter that restarts at zero never repeats a (key, nonce) pair. The derivation context is public. Security comes from the independent secret K_e and the KMS/HSM authorization boundary, not from hiding the graph digest or epoch identifier. Derivation should occur inside the KMS/HSM boundary so that K_e is never exported; where a KMS cannot perform the chosen KDF internally, the design must state where K_e is exposed and treat that component as part of the key authority's trust boundary.

Under an ideal independent-key model, $I(K_e; K_{e+1}) = 0$. In implementation terms this should be called cross-epoch key independence or epoch key isolation, not session forward secrecy. Computational separation depends on the entropy source, KMS/HSM isolation, authorization policy, and independent key-generation events. Deleting a live epoch key does not imply every escrow, backup, forensic, or recovery copy has vanished; those domains require separate governance.

$$\text{Ideal cross-epoch independence: } I(K_e; K_{e+1}) = 0 \quad (27)$$

12.2 AEAD-protected mission-domain transfer

State-transfer objects should use authenticated encryption with associated data (AEAD). AES-GCM is one standardized option; NIST SP 800-38D requires IV uniqueness within a key context and protects both ciphertext and associated-data authenticity [52]. NIST is revising SP 800-38D, including removing authentication tags shorter than 96 bits and clarifying IV-construction guidance [53]; MIAM profiles use 128-bit tags. The preferred profile allocates a dedicated transfer key per epoch, service, purpose, and SCF instance, with a 96-bit nonce from an instance-local monotonic counter. Because each SCF instance derives its transfer key with a never-reused inst_j in Equation (26), counter state never needs to survive a crash or be shared across replicas; a restarted instance receives a new identifier and key rather than resuming a counter. Deployments that cannot guarantee per-instance keys should use a nonce-misuse-resistant AEAD such as AES-GCM-SIV [54].

$$\text{nonce} = \text{Encode}_{96}(\text{instance_local_counter}) \quad (28)$$

$$(C, T) = \text{AEAD_Enc}(K_{\text{state}}, \text{nonce}, P, \text{AAD}) \quad (29)$$

$$\text{AAD} = \langle \text{key_id}, \text{src_epoch}, \text{dst_epoch}, \text{direction}, \text{manifest_digest}, \\ \text{service_id}, \text{schema_id}, \text{object_id}, \text{sequence} \rangle \quad (30)$$

The manifest digest commits to both the source and target graph digests, the graph delta, and inst_j , so each object is bound to one specific transition rather than to one graph. The SCF selects each decryption key from the signed transition manifest, never from a field in the object itself. AES-GCM is not key-committing, and a receiver that chooses among candidate keys based on attacker-controlled input can be exposed to partitioning-oracle attacks [55]; binding the key identifier in the AAD and fixing the key by manifest removes that choice. The SCF decrypts and authenticates under a source-epoch transfer key, rejects any object whose source epoch, sequence, graph context, or authorization is not currently permitted, applies the compiled state policy and target-specific validators, and re-encrypts accepted objects under an independently derived target-epoch transfer key. Validation failure causes quarantine, repair, or transition abort according to Section 11. Durable business data normally remains under a separate long-lived data-encryption/key-encryption hierarchy so retirement of transient epoch keys cannot destroy required records.

12.3 Workload identity, mTLS, and crypto agility

Workload identities should be short-lived and epoch-aware. SPIFFE provides standardized workload identities and short-lived X.509 SVIDs that can establish mutual TLS between workloads [56]. If all epochs share one trust domain and one issuing CA, epoch isolation rests entirely on the authorization gate. SPIFFE itself defines workload identity and SVID delivery; it does not define epochs. The following rules are therefore a MIAM identity profile layered on SPIFFE, not SPIFFE-provided semantics. The epoch is encoded in the workload identity (for example, as a path segment of the SPIFFE ID), and, where the identity system supports it, each epoch's SVIDs are issued from a distinct intermediate CA under the same trust-domain root. During overlap, target-epoch services accept only target-epoch identities for target-epoch service boundaries, and the SCF channel is the only path that accepts both. At retirement, the source intermediate is

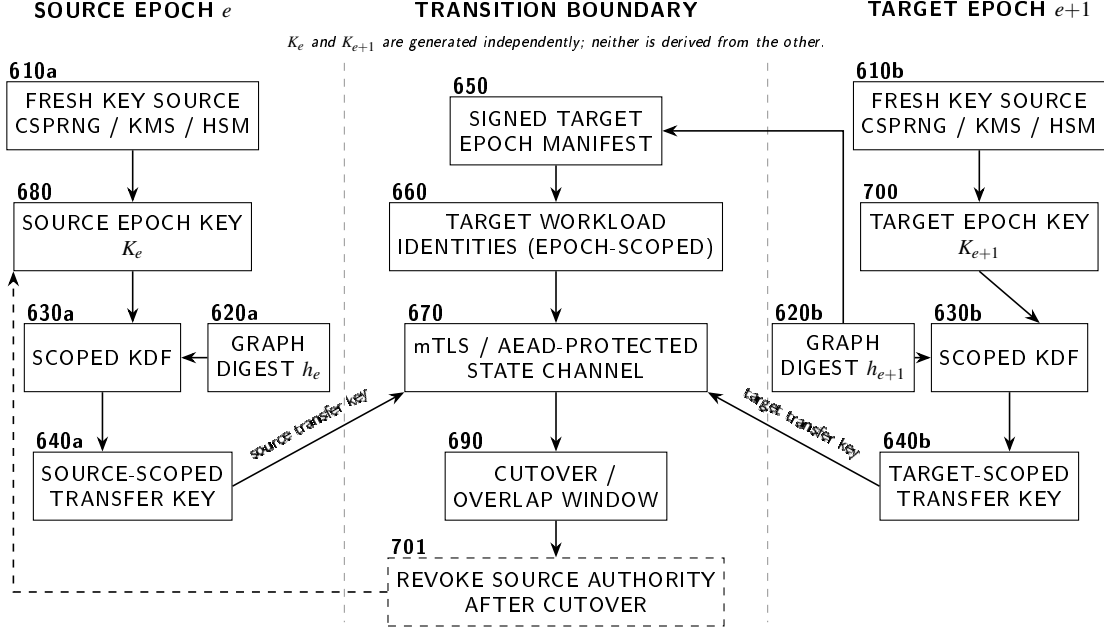


Figure 7: Cryptographic epoch isolation: independent epoch key generation, graph binding, scoped derivation, epoch-scoped workload identity, protected channels, and source-epoch revocation. Suffixes *a* and *b* denote the same element type in the source and target epochs.

removed from the published trust bundle and its signing key is destroyed. A rollback never reactivates a retired epoch: it instantiates the known-good graph as a new epoch with a new intermediate. If a trust-bundle update fails to propagate, affected target services continue to reject source identities, because the authorization gate below does not depend on bundle state. MIAM does not rely on certificate revocation alone for immediate epoch separation: an authorization gate checks the signed active/overlap epoch manifest and rejects a source-epoch principal at target-epoch service boundaries even if its certificate has not yet expired, and the gate fails closed if the manifest cannot be verified. Long-lived connections are drained, sent an application/protocol reconnect signal where available, or pinned to the source graph until an explicit deadline; they are not silently migrated as authenticated target-epoch sessions. Direct source-to-target service calls remain denied except through the SCF transfer channel.

Algorithm identifiers belong in a versioned cryptographic policy rather than in the core control logic. NIST’s 2026 crypto-agility guidance emphasizes the ability to replace algorithms while maintaining ongoing operations [57]. A future enterprise profile may support NIST-standardized ML-KEM for quantum-resistant key establishment and ML-DSA for signatures, but post-quantum algorithms are implementation options, not the central concept [58, 59].

13. Enterprise Implementation Blueprint

A prototype can be implemented with current cloud-native primitives while keeping the design vendor neutral. Kubernetes Services and EndpointSlices can provide a stable logical name while backend instances change; Istio VirtualServices can decouple client destinations from workload routing; SPIFFE/SPIRE can supply workload identity; Open Policy Agent/Gatekeeper can enforce admission constraints; and OpenTelemetry traces can reconstruct service-call paths for the observability graph. These products are examples, not requirements of the architecture [60, 61, 62, 63].

The Mission Contract Registry stores versioned interface schemas, invariants, SLOs, data-residency con-

straints, and capability-state contracts. The Observability Graph Builder combines traces with deployment and policy metadata. The Candidate Generator applies only certified grammar rules. The scorer evaluates prerequisite actionability, path survival, absolute exposure, operational cost, variant leakage, and transition detectability. The Verification Authority vetoes unsafe candidates. The Epoch Orchestrator stages, drains, cuts over, and rolls back. The Identity/Key Authority supplies epoch-scoped authority. The SCF enforces the compiled state policy. The Evidence Ledger records signed graph digests, verification evidence, estimates, and transition outcomes, and should be append-only and tamper-evident.

13.1 Transition eligibility and stateful workloads

MIAM must classify workload boundaries before attempting a morph. Class M0 workloads are stateless or externally stateful and may transition online. Class M1 workloads have bounded registered mission-domain state and may transition through the SCF at a defined safe point. Class M2 workloads contain transactions, streams, leases, queues, or long-lived sessions and require a specific drain/fencing/resumption protocol before they are eligible. Class M3 workloads have unbounded in-process state, unsafe external side effects, unsupported transaction semantics, or unverifiable dependencies and are pinned to the current graph; MIAM may route around or isolate them but does not dynamically repartition them.

For M1/M2 transitions, the orchestrator establishes a write fence or lease epoch, stops admission of new work to affected source capabilities, drains or resolves in-flight operations, captures registered queue offsets and idempotency state where required, transfers validated mission-domain state, activates target ownership, and only then moves the stable interface. Distributed locks are not copied; they are reacquired under a new fencing token after source ownership expires. Queue consumers use explicit partition/offset handoff and deduplication rules. Long-running connections are drained or resumed at the application layer. Durable databases remain external to the morph unless a separately certified storage-migration protocol is invoked.

13.2 Brownfield integration

A commercially credible brownfield deployment does not require a legacy application to become fully morphable. The first integration layer can sit behind an existing gateway or load balancer, build a capability/dependency map from telemetry, and manage only a small set of pre-certified stateless or bounded-state components. Legacy cores, databases, proprietary appliances, and non-morphable transaction engines remain pinned. Over time, stable adapters can expose selected capabilities as controlled variation points. This incremental model resembles a strangler migration in deployment shape but uses security-driven graph selection rather than requiring wholesale application rearchitecture.

14. Reference Algorithms

14.1 Morph selection

Algorithm 1: SELECT-MORPH(current G_e , mission M , knowledge $K_{\leq e}$, capability-state map Σ , reference graph G_{ref} , policy P) Precondition, checked when the pool, policy, or schedule changes: the half-time bound T_{max} and the recurrence-floor bound R_{max} of Section 7 hold; otherwise the policy is rejected and no transition runs.

1. $\Omega \leftarrow \text{GenerateCandidates}(G_e, \text{CertifiedGrammar}, P)$
2. $\Omega \leftarrow \{G \in \Omega : \text{StaticConstraints}(G, M, P) = \text{PASS}\}$
3. **for each** $G \in \Omega$:

4. Evaluate prerequisite predicates over $K_{\leq e}$; $r \leftarrow \text{RTR}(K_{\leq e}, G)$; $q \leftarrow \text{PathSurvival}(K_{\leq e}, G)$
5. $x \leftarrow \text{Exp}_{X_v}(G)$ from the attack graph of G and the privilege-concentration map under the pinned exposure-model version X_v
6. $\Delta \leftarrow \text{Diff}(G_e, G)$; $P_{\text{state}} \leftarrow \text{CompileStatePolicy}(\Delta, M, \Sigma)$
7. $f \leftarrow \text{VariantLeakage}(G)$; $z \leftarrow \text{TransitionDetectability}(G_e, G)$; $c \leftarrow \text{MutationCost}(G)$; $l \leftarrow \text{SLORisk}(G)$
8. $j \leftarrow \lambda_R r + \lambda_P q + \lambda_X x + \lambda_C \hat{c} + \lambda_L \hat{l} + \lambda_F f + \lambda_V z + \lambda_S \widehat{\text{StateRisk}}(P_{\text{state}})$
9. **if** $\text{VerifyMission}(G, M) = \text{PASS}$ **and** $\text{VerifyStatePolicy}(P_{\text{state}}) = \text{PASS}$ **and** $r \leq 1 - r_{\min}$ **and** $x \leq \text{Exp}_{X_v}(G_{\text{ref}}) + \delta_X$ **and** $D_{\text{arch}}(G_e, G) \geq d_{\min}$: retain $(G, \Delta, P_{\text{state}}, j)$
10. **if** fewer than $m_{\min}$ candidates are retained for longer than the declared interval: raise a candidate-exhaustion alert
11. **return** $\arg \min_j$ over retained candidates; if none, KEEP_CURRENT or SAFE_FALLBACK

14.2 Verified epoch transition

Algorithm 2: TRANSITION(source G_e , selected G_n , graph delta Δ , compiled state policy P_{state})

1. Verify the signed target manifest, then generate an independent target epoch key K_n inside the KMS/HSM.
2. Canonicalize and sign the target graph manifest including h_n , Δ , P_{state} , mission version, and policy version.
3. Issue epoch-scoped target workload identities and scoped keys authorized only for target-epoch boundaries and the dedicated SCF channel.
4. Instantiate G_n dark and pre-warm it; execute contract, policy, consistency, content-safety, exposure, and SLO gates.
5. Instantiate a fresh SCF from a signed image; the KMS/HSM issues a never-reused instance identifier inst_j , binds it into the manifest, and derives the per-instance transfer keys.
6. Enter transition safe point: fence affected writes/admission, revoke source write authority, drain or checkpoint eligible in-flight work, freeze registered ownership metadata, take the fenced snapshot, and record its digest in the manifest. Start the overlap deadline T_{ov} .
7. Export typed mission-domain state; the SCF authenticates, enforces P_{state} , validates schema, prove-nance, replay, and business invariants, and re-encrypts accepted objects for G_n . Route failed mission-required objects to quarantine-and-repair within the retry budget; otherwise abort and restore.
8. Rehydrate target; reacquire leases/locks with new fencing epochs; restore approved queue offsets/idempotency state; run reconciliation.
9. Shift the stable interface gradually while source and target remain cross-epoch isolated; monitor and roll back on invariant violation or on expiry of T_{ov} .
10. End overlap; reject source principals at target boundaries, revoke/expire source identities and the source intermediate CA bundle, delete live source keys, destroy the SCF instance, and retire/quarantine the source runtime.
11. Record transition evidence, actual prerequisite replay outcomes, pinning duration if any, and update calibrated actionability and recurrence estimates.

15. Evaluation Methodology

The research hypothesis must be falsifiable. The first experiment uses one application implemented as a capability library over a public microservice benchmark such as TrainTicket or DeathStarBench [64, 65]. Using TrainTicket also permits direct comparison with recent cloud-native MTD results on the same application [20]. The certified pool size m is an experimental factor, swept over at least $m \in \{4, 8, 16\}$, because Section 6.6 shows that small pools impose high recurrence floors.

Identical business workloads, vulnerability sets, and attacker objectives are exercised under the following arms:

- (a) static architecture;
- (b) address/port MTD;
- (c) placement/container MTD;
- (d) *rejuvenation-only*: the same graph redeployed each epoch with fresh epoch keys, fresh workload identities, and the SCF, but no graph change;
- (e) *morph-only*: graph morphing without epoch key or identity isolation;
- (f) full MIAM;
- (g) *boundary-only*: full MIAM restricted to morphs that change only the capability-to-service partition, holding implementation family, placement, routing, and protocol fixed.

Arms (d), (e), and (g) are ablations. The difference between (f) and (d) isolates the contribution of service-graph morphing from the contribution of rejuvenation and credential rotation, which prior work already provides; the difference between (f) and (e) isolates the contribution of epoch isolation; and the difference between (g) and (d) isolates the contribution of service-boundary repartitioning, which is the paper’s narrow distinction, from other graph changes that prior MTD work already covers. A result in which (f) does not improve on (d) for structure-bound prerequisites would falsify MIAM’s central hypothesis, and a result in which (g) does not improve on (d) would falsify the specific value of boundary repartitioning. To avoid confounds, all dynamic arms use the same transition cadence, the same pre-warming and warm-up procedure, and the same per-transition resource budget, and transition downtime is measured and reported per arm.

Each attack prerequisite is represented by a machine-readable predicate and a replay action so the same prerequisite can be evaluated against the source and target graphs without changing its definition after results are observed. Attack graphs for exposure scoring can be generated with logic-based tools such as MulVAL [41]. Attackers are modeled at four levels: non-adaptive; adaptive-remapping; persistent, accumulating knowledge across epochs; and MTD-aware, detecting transition events from observable footprints and timing activity to them [17]. Three targeted adversary tests are added: an adversarial-estimator test in which up to f corroborating telemetry sources report authenticated but false values; an overlap test in which a source-resident attacker acts continuously from before the fence through cutover; and a candidate-exhaustion test in which the attacker induces load or resource pressure to shrink the feasible set.

Primary security outcomes are RTR, attack-path survival, and absolute exposure measured from replayable prerequisite predicates and attack graphs, reported separately for epoch-bound and structure-bound prerequisites. Secondary outcomes are the measured recurrence floor as a function of m , time to remap, old-epoch credential acceptance, forbidden-state carryover, active-graph fingerprint accuracy, transition detectability F_{evt} , pinning duration under planted objects, transition latency/error/throughput, mission-consistency violations, rollback rate, and CPU/memory/network/control-plane cost. Each metric must name its unit, sampling window, and estimator before data collection. Where applicable, composite

reporting should follow existing attacker-aware MTD evaluation practice so results are comparable across studies [21].

The experiment should use paired attack/graph seeds across arms and vary morph intervals over a predeclared range. A pilot estimates variance; the final number of trials is then set by power analysis, with at least 100 source-to-target transitions per principal condition as an initial floor rather than a guarantee of power. Report 95% confidence intervals. Use paired permutation or mixed-effects models for RTR/PSR, survival analysis for time-to-remap, and block bootstrap intervals for latency/throughput because request samples are autocorrelated. Pre-register falsification gates publicly before observing outcomes. The paper deliberately does not invent security gains: until a prototype is measured, all claimed benefit remains hypothesis rather than result.

16. Computational Checks and Worked Example

The equations were evaluated numerically. The reproduction script `miam_checks.py` accompanies this preprint and is published in the public repository at <https://github.com/codethor0/miam>. As a simple schedule example, assume deterministic morph period $\tau = 300$ s and empirically measured prerequisite retention $\rho = 0.25$. Equation (16) gives a knowledge-retention half-time $T_{50,\text{det}}$ of 150 s. If morph arrivals are Poisson with mean interval 300 s, Equation (18) gives $T_{50,\text{Pois}} \approx 277.26$ s. These differ because deterministic morphs guarantee a reduction at fixed intervals whereas the Poisson model averages over random event counts.

To make actionability operational rather than arbitrary, consider a synthetic replay harness with $N = 20$ repetitions for five fixed prerequisite predicates. Suppose the predicates succeed against a candidate graph $[2, 8, 0, 5, 16]$ times, giving $v = [0.1, 0.4, 0, 0.25, 0.8]$. With weights $w = [5, 3, 2, 4, 1]$ and possession likelihoods $c = [0.9, 0.8, 0.7, 0.6, 0.5]$, the RTR numerator is 2.41, denominator 11.2, and $\text{RTR} = 0.21518$. These replay counts are synthetic arithmetic fixtures, not measured MIAM efficacy. For three illustrative paths under the explicit independence approximation, actionabilities $[0.1, 0.4, 0.25]$, $[0.4, 0.8]$, and $[0.1, 0.8]$ yield path survivals 0.01, 0.32, and 0.08; with path weights $[5, 3, 2]$, $\text{PSR} = 0.117$.

For the service-boundary morph of Figure 5, the source partition is $\{\{A, B\}, \{C, E\}, \{D\}\}$ and the target partition is $\{\{A, B, D\}, \{C\}, \{E\}\}$ over $n = 5$ capability units. Then $H(\Pi) = 1.52193$ bits, $H(\Pi') = 1.37095$ bits, the joint entropy is 1.92193 bits, $\text{VI} = 0.95098$ bits, and $D_\Pi = 0.95098 / \log_2 5 = 0.40956$. The attacker-completion example with mean $T_A = 1200$ s, $\lambda = 1/300 \text{ s}^{-1}$, and $\rho = 0.25$ yields $b/[b + \lambda(1 - \rho)] = 0.25$.

For the recurrence model, the four-graph pool with no self-transition gives the eigenvalue 1 and the eigenvalue $-1/3$ with multiplicity three. A structure-bound prerequisite valid only in the source graph has expected actionability 1, 0, 0.3333, 0.2222, 0.2593, 0.2469, 0.2510, 0.2497, and 0.2501 over morphs 0 through 8, converging to 0.25. Under Poisson morphs with $\lambda = 1/300 \text{ s}^{-1}$, its expected actionability is $1/4 + \frac{3}{4}e^{-4\lambda t/3}$, so its half-time is $3 \ln 3 / (4\lambda)$, approximately 247.19 s. A prerequisite valid in two of the four graphs converges to 0.5 and has no half-time.

A Monte Carlo check with 120,000 Poisson samples at the analytical half-time produced mean retention 0.4980 with a 95% confidence interval of ± 0.0023 for $\rho = 0.25$ and $\lambda = 1/300 \text{ s}^{-1}$, consistent with the closed-form value of 0.5. These calculations validate internal arithmetic only, not the underlying security assumptions.

17. Enterprise Adoption Path

A major enterprise is unlikely to accept arbitrary self-rewriting production infrastructure. Adoption becomes plausible when MIAM is introduced as a bounded resilience control with auditable variants, deterministic

rollback, stable external contracts, policy-as-code, and measurable value. The deployment path should therefore begin with pre-certified variants and manual/risk-triggered transitions, then automate scoring and canary cutovers, and only later permit closed-loop generation within a verified grammar.

Phase 0, offline digital twin: model the capability graph, mission contract, attack prerequisites, candidate variants, and recurrence floors without production mutation.

Phase 1, pre-certified pool: deploy hand-reviewed graph variants and exercise controlled cutovers in staging.

Phase 2, production bounded morphing: automate target selection, canary, rollback, epoch identity, and SCF under strict change windows.

Phase 3, risk-adaptive morphing: use calibrated prerequisite-actionability evidence, authenticated telemetry, and bounded trigger logic to choose transitions and timing.

Phase 4, cross-substrate resilience: incorporate multi-cluster/cloud/substrate diversity only after application-level invariants are proven.

The business case should be measured against threat classes that genuinely depend on mapping and dwell time. For low-latency destructive exploits, a morph may arrive too late; for long-dwell lateral movement and credential reuse, the architecture may have stronger leverage. The system should be positioned as a complement to hardening and zero trust, not as a replacement for them.

18. Limitations, Failure Modes, and Open Research Problems

- Common-mode vulnerabilities survive morphs when every variant shares the same vulnerable library, API logic, secret, or trust decision.
- Finite certified pools impose a nonzero recurrence floor on structure-bound knowledge. An attacker who maps every graph in a small pool gains durable knowledge that no morph rate can remove; pool size and diversity are first-order security parameters, not operational details.
- Breaking old knowledge is not the same as reducing attack surface. Without the exposure bound, an optimizer can select graphs that are easier to attack from scratch.
- Much of the benefit of any epoch transition may come from rejuvenation and credential rotation rather than graph change. Until the ablation of Section 15 is run, the specific contribution of service-graph morphing is unmeasured.
- Control-plane compromise is catastrophic; the mutation authority requires stronger isolation, authentication, code-signing, and change governance than ordinary workloads.
- Observability continuity is difficult: renaming/repartitioning services can fragment incident timelines unless the evidence ledger preserves stable capability identities.
- Stateful systems with long-running transactions, distributed locks, or external side effects require explicit handoff semantics; not every service is morphable at arbitrary times.
- Candidate scoring can be gamed if telemetry is poisoned. Provenance authenticates the source of a report, not its truth; the corroboration assumption of Section 4 (at most f of k sources adversarial) is a stated assumption, and an attacker who controls more sources than that can steer selection.

- Transition events can be observable even when the active variant is not. An MTD-aware attacker can time activity to epoch boundaries.
- The SCF is both a chokepoint and a pinning vector. Its hardening, quarantine-and-repair path, and in-place containment are part of the security model, but they provide detection and containment rather than guaranteed forward progress; an attacker who can continuously inject distinct mission-required failures can prevent service-graph morphing indefinitely.
- An attacker who can shrink the feasible candidate set through load, SLO pressure, or resource exhaustion can steer selection toward variants it has already mapped. The candidate-exhaustion alert detects this but does not prevent it.
- High morph frequency increases cost, cache churn, cold starts, certificate/key operations, and operational complexity. Security value must exceed these costs.
- Formal mission equivalence is limited. Production implementations will combine formal invariants with test evidence rather than prove arbitrary program equivalence.
- Mission-domain state can carry attacks. Schema-valid records, workflow data, serialized content, or persistent configuration can still trigger target vulnerabilities; the SCF reduces this risk through graph-specific validation but cannot prove arbitrary business content safe.
- Overlap is a security boundary, not merely an availability convenience. Source identities and sessions must not gain target-epoch authority; cross-epoch workload communication is denied except through explicitly authorized transition channels.
- Some attacker knowledge is intentionally invariant: external contracts, business rules, durable schemas, common dependencies, and control-plane entry points may remain learnable across all epochs. MIAM can only decay knowledge tied to properties that the admissible graph family is allowed to change.
- Telemetry poisoning and control-plane compromise can turn adaptation into an attack primitive. Signed transition manifests, provenance, multi-source corroboration, trigger rate limits, hysteresis, separated verification authority, and a known-good fallback are part of the security model, not optional operational polish.
- The current work is a design and mathematical model without experimental data. Security efficacy and overhead remain open questions.

19. Claim-Oriented Distinction and Defensive Publication

The search did not identify a single reviewed reference that clearly discloses the full integrated transition protocol in one closed loop; this is not a legal novelty conclusion and the search is not exhaustive. Each ingredient taken alone is established: architecture-level MTD [24], attack-graph-driven MTD and reconfiguration selection [12, 13, 30], microservice MTD [18, 14, 20], and state migration as an MTD action [22]. The distinction claimed here is only their specific integration. The distinction is strongest when framed as the technical relationship among: (1) mission-constrained service-boundary/dependency transformation; (2) machine-evaluable prerequisite actionability against candidate graphs, bounded by absolute exposure; (3) a graph delta that maps source capability/state ownership to target ownership; (4) compilation of a state-transition policy from that delta and capability-state contracts; (5) verified transfer of only permitted mission-domain state through an isolated continuity boundary; and (6) target-epoch authority that remains isolated during overlap and replaces source authority after cutover.

The weakest framing remains “change architecture often.” Patents and papers already occupy that territory. The stronger framing is a concrete distributed transition mechanism in which prerequisite evaluation materially constrains target selection and the same graph transformation produces the state-policy and authority boundaries used during cutover. The cryptographic primitives, scoring arithmetic, Kubernetes mechanisms, and service-mesh products are supporting embodiments rather than the central concept by themselves.

This paper is released publicly as independent open-source cybersecurity research. Appendix A is included as a reference claim set so that the integrated mechanism is described with claim-level precision and enters the public record as prior art. Its purpose is to support open review, reimplement, and extension by the security community, not to assert exclusive rights.

20. Conclusion

MIAM reframes moving-target defense from moving infrastructure toward reducing the continued actionability of an attacker’s learned application model. The design does not assume that every system can morph, that mission-domain state is inherently safe, or that breaking old knowledge is the same as reducing exposure. Instead, it operationalizes prerequisite actionability, bounds absolute exposure, derives state policy from the source-to-target graph delta, isolates source and target authority during overlap, and excludes non-morphable workloads from unsafe transitions. The recurrence-aware retention model shows that a finite variant pool leaves a knowledge floor that must be measured and bounded rather than assumed away. The mathematical models define hypotheses and decision variables rather than security guarantees. Broad dynamic-architecture claims remain crowded, but the integrated transition protocol is specific enough to prototype and to evaluate. The next technical gate is implementation: build a capability library over a public microservice benchmark, a certified pool of graphs of varying size, predicate/replay evaluators, an attack-graph exposure scorer, graph-delta state policy, epoch identities/keys, and a red-team harness, then measure whether MIAM reduces prerequisite reuse beyond both conventional MTD and rejuvenation alone without increasing exposure or violating mission and SLO constraints. Feedback, replication, and critique are welcome.

Availability

This preprint is released as independent open-source cybersecurity research. The paper source, the reproduction script for Section 16, and future revisions are published at <https://github.com/codethor0/miam>. Corrections, replications, prototype implementations, and critiques are welcome through the contact details on the first page.

References

- [1] R. Zhuang, S. Zhang, A. Bardas, S. A. DeLoach, X. Ou, and A. Singhal, “Investigating the Application of Moving Target Defenses to Network Security,” NIST / ISRCs, 2013. <https://www.nist.gov/publications/investigating-application-moving-target-defenses-network-security>
- [2] R. Zhuang, S. A. DeLoach, and X. Ou, “Towards a Theory of Moving Target Defense,” in *Proc. First ACM Workshop on Moving Target Defense (MTD ’14)*, pp. 31–40, 2014.
- [3] J.-H. Cho, D. P. Sharma, H. Alavizadeh, S. Yoon, N. Ben-Asher, T. J. Moore, D. S. Kim, H. Lim, and F. F. Nelson, “Toward Proactive, Adaptive Defense: A Survey on Moving Target Defense,” *IEEE Communications Surveys & Tutorials* 22(1), 2020. <https://arxiv.org/abs/1909.08092>

- [4] S. Sengupta, A. Chowdhary, A. Sabur, A. Alshamrani, D. Huang, and S. Kambhampati, "A Survey of Moving Target Defenses for Network Security," *IEEE Communications Surveys & Tutorials* 22(3), 1909–1941, 2020. <https://doi.org/10.1109/COMST.2020.2982955>
- [5] E. Yuan, S. Malek, B. Schmerl, D. Garlan, and J. Gennari, "Architecture-based Self-protecting Software Systems," in *Proc. 9th Int. ACM SIGSOFT Conf. on Quality of Software Architectures (QoSA)*, pp. 33–42, 2013.
- [6] F. Damiani, L. Padovani, and I. Schaefer, "A Formal Foundation for Dynamic Delta-Oriented Software Product Lines," GPCE 2012. <https://doi.org/10.1145/2371401.2371403>
- [7] S. A. Alhozaimy, D. A. Menascé, and M. Albanese, "Design and Modeling of Moving Target Defense in Workflow-Based Applications," *Cluster Computing* 27, 945–958, 2024. <https://doi.org/10.1007/s10586-023-03998-9>
- [8] D. Ergenç, F. Schneider, P. Kling, and M. Fischer, "Moving Target Defense for Service-oriented Mission-critical Networks," 2023. <https://arxiv.org/abs/2303.09893>
- [9] J. Yackoski, J. Li, S. A. DeLoach, and X. Ou, "Mission-oriented Moving Target Defense Based on Cryptographically Strong Network Dynamics," in *Proc. 8th Cyber Security and Information Intelligence Research Workshop (CSIIRW)*, 2013. <https://doi.org/10.1145/2459976.2460040>
- [10] H. Li, W. Shen, and Z. Zheng, "Spatial-Temporal Moving Target Defense: A Markov Stackelberg Game Model," AAMAS 2020. <https://arxiv.org/abs/2002.10390>
- [11] D. P. Sharma, S. Y. Enoch, J.-H. Cho, T. J. Moore, F. F. Nelson, H. Lim, and D. S. Kim, "Dynamic Security Metrics for Software-Defined Network-based Moving Target Defense," *Journal of Network and Computer Applications* 170, 102805, 2020. <https://doi.org/10.1016/j.jnca.2020.102805>
- [12] S. Yoon, J.-H. Cho, D. S. Kim, T. J. Moore, F. Free-Nelson, and H. Lim, "Attack Graph-Based Moving Target Defense in Software-Defined Networks," *IEEE Transactions on Network and Service Management* 17(3), 1653–1668, 2020. <https://doi.org/10.1109/TNSM.2020.2987085>
- [13] V. Zangeneh and M. Shajari, "A Cost-Sensitive Move Selection Strategy for Moving Target Defense," *Computers & Security* 75, 72–91, 2018. <https://doi.org/10.1016/j.cose.2017.12.013>
- [14] X. Liu, S. Zhang, S. Huo, and K. Shang, "Microservice Moving Target Defense Strategy Based on Adaptive Genetic Algorithm," *Computer Science* 50(9), 82–89, 2023. <https://doi.org/10.11896/jsjx.221000199>
- [15] T. Hobson, H. Okhravi, D. Bigelow, R. Rudd, and W. Streilein, "On the Challenges of Effective Movement," in *Proc. First ACM Workshop on Moving Target Defense (MTD '14)*, pp. 41–50, 2014.
- [16] S. Sengupta and A. Chowdhary, " ϵ -Indistinguishability in Moving Target Defense: Framework, Algorithms, and Cloud Case Studies," arXiv:2607.13440, 2026. <https://arxiv.org/abs/2607.13440>
- [17] T. Moghaddam, G. Yang, C. Thapa, S. Camtepe, and D. S. Kim, "MTDSense: AI-Based Fingerprinting of Moving Target Defense Techniques in Software-Defined Networking," arXiv:2408.03758, 2024. <https://arxiv.org/abs/2408.03758>
- [18] K. A. Torkura, M. I. H. Sukmana, A. Kayem, F. Cheng, and C. Meinel, "A Cyber Risk Based Moving Target Defense Mechanism for Microservice Architectures," IEEE BDCloud 2018. <https://doi.org/10.1109/BDCloud.2018.00137>
- [19] K. R. González Pineda, F. Rodríguez Zamora, and E. G. Barrantes Sliesarieva, "Moving Target Defense for Diversification of Microservices on Kubernetes," Universidad de Costa Rica, 2023. <https://hdl.handle.net/10669/104575>
- [20] S. Ma, L. Ding, T. Zhou, L. Chang, and L. Li, "Moving Target Defense Strategies Against Lateral Movement in Cloud-Native Environments," *Future Generation Computer Systems* 185, 108676, 2026. <https://doi.org/10.1016/j.future.2026.108676>
- [21] M. Farhad, M. A. Tahera, P. J. Thapa, S. Dass, and B. Acharya, "MTD-Playground: An Attacker-Aware Evaluation Framework for Network Moving Target Defense," arXiv:2607.12199, 2026. <https://arxiv.org/abs/2607.12199>
- [22] A. Mamaril, R. Kolodziejczyk, W. Soussi, and G. Gür, "Exploring Live Payload Migrations for MTD in Microservices Architecture," in *2024 IEEE 99th Vehicular Technology Conference (VTC2024-Spring)*, 2024. <https://doi.org/10.1109/VTC2024-Spring62846.2024.10683429>

- [23] B. Cox, D. Evans, et al., “N-Variant Systems: A Secretless Framework for Security through Diversity,” 15th USENIX Security Symposium, 2006.
- [24] B. Schmerl, J. Cámara, G. A. Moreno, D. Garlan, and A. Mellinger, “Architecture-Based Self-Adaptation for Moving Target Defense,” Technical Report CMU-ISR-14-109, Carnegie Mellon University, 2014.
- [25] J. O. Kephart and D. M. Chess, “The Vision of Autonomic Computing,” *Computer* 36(1), 41–50, 2003. <https://doi.org/10.1109/MC.2003.1160055>
- [26] Y. Huang, C. Kintala, N. Kolettis, and N. D. Fulton, “Software Rejuvenation: Analysis, Module and Applications,” FTCS-25, 381–390, 1995. <https://doi.org/10.1109/FTCS.1995.466961>
- [27] G. Candea, S. Kawamoto, Y. Fujiki, G. Friedman, and A. Fox, “Microreboot: A Technique for Cheap Recovery,” OSDI 2004.
- [28] M. K. Reiter and K. P. Birman, “How to Securely Replicate Services,” *ACM Transactions on Programming Languages and Systems* 16(3), 986–1009, 1994. <https://doi.org/10.1145/177492.177745>
- [29] US10447710B1, “Self-shielding dynamic network architecture.” <https://patents.google.com/patent/US10447710B1/en>
- [30] K. Norrman, J. Cederberg, and M. Näslund, US8646085B2, “Apparatus for reconfiguration of a technical system based on security analysis and a corresponding technical decision support system and computer program product,” 2014. <https://patents.google.com/patent/US8646085B2/en>
- [31] US10958478B2, “Resilient polymorphic network architectures.” <https://patents.google.com/patent/US10958478B2/en>
- [32] US11005886B2, “Moving target defense with network level changes providing substantially continuous access to applications.” <https://patents.google.com/patent/US11005886B2/en>
- [33] US11729221B1, “Reconfigurations for network devices.” <https://patents.google.com/patent/US11729221B1/en>
- [34] US11748490B2, “Computer system with moving target defenses against vulnerability attacks.” <https://patents.google.com/patent/US11748490B2/en>
- [35] US12124887B2, “Microservice measurement and merging.” <https://patents.google.com/patent/US12124887B2/en>
- [36] US12541603B2, “Architecture swapping for workspaces.” <https://patents.google.com/patent/US12541603B2/en>
- [37] EP4046020A1, “Methods, apparatus and systems for decomposing mobile applications into micro-services at runtime for distributed execution.” <https://patents.google.com/patent/EP4046020A1/en>
- [38] CN121116381A, “Cloud-native microservice decomposition and refactoring method based on runtime behavior.” <https://patents.google.com/patent/CN121116381A/en>
- [39] CN120743522A, “Service reconstruction method, device, electronic equipment, storage medium and program product.” <https://patents.google.com/patent/CN120743522A/en>
- [40] NIST SP 800-207A, “A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Cloud Environments,” 2023. <https://csrc.nist.gov/pubs/sp/800/207/a/final>
- [41] X. Ou, S. Govindavajhala, and A. W. Appel, “MulVAL: A Logic-based Network Security Analyzer,” 14th USENIX Security Symposium, 2005.
- [42] P. K. Manadhata and J. M. Wing, “An Attack Surface Metric,” *IEEE Transactions on Software Engineering* 37(3), 371–386, 2011. <https://doi.org/10.1109/TSE.2010.60>
- [43] M. Meilă, “Comparing Clusterings: An Information Based Distance,” *Journal of Multivariate Analysis* 98(5), 873–895, 2007. <https://doi.org/10.1016/j.jmva.2006.11.013>
- [44] NIST SP 800-57 Part 1 Rev. 5, “Recommendation for Key Management: Part 1, General,” 2020. <https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final>
- [45] NIST SP 800-133 Rev. 2, “Recommendation for Cryptographic Key Generation,” 2020. <https://csrc.nist.gov/pubs/sp/800/133/r2/final>

- [46] A. Rundgren, B. Jordan, and S. Erdtman, RFC 8785, “JSON Canonicalization Scheme (JCS),” 2020. <https://www.rfc-editor.org/info/rfc8785>
- [47] C. Bormann and P. Hoffman, RFC 8949, “Concise Binary Object Representation (CBOR),” 2020, Section 4.2. <https://www.rfc-editor.org/info/rfc8949>
- [48] NIST FIPS 180-4, “Secure Hash Standard.” <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>
- [49] H. Krawczyk and P. Eronen, RFC 5869, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” 2010. <https://www.rfc-editor.org/info/rfc5869>
- [50] NIST SP 800-108 Rev. 1 (updated February 2024), “Recommendation for Key Derivation Using Pseudorandom Functions.” <https://doi.org/10.6028/NIST.SP.800-108r1-upd1>
- [51] NIST SP 800-56C Rev. 2, “Recommendation for Key-Derivation Methods in Key-Establishment Schemes,” 2020. <https://doi.org/10.6028/NIST.SP.800-56Cr2>
- [52] NIST SP 800-38D, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” 2007. <https://csrc.nist.gov/pubs/sp/800/38/d/final>
- [53] NIST SP 800-38D Rev. 1, Second Pre-Draft Call for Comments, June 2026. <https://csrc.nist.gov/pubs/sp/800/38/d/r1/2prd>
- [54] S. Gueron, A. Langley, and Y. Lindell, RFC 8452, “AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption,” 2019. <https://www.rfc-editor.org/info/rfc8452>
- [55] J. Len, P. Grubbs, and T. Ristenpart, “Partitioning Oracle Attacks,” 30th USENIX Security Symposium, 2021.
- [56] SPIFFE, “Workload API” and SVID specifications. https://spiffe.io/docs/latest/spiffe-specs/spiffe_workload_api/
- [57] NIST CSWP 39 upd. 1, “Considerations for Achieving Crypto Agility: Strategies and Practices,” 2026. <https://csrc.nist.gov/pubs/cswp/39/upd1/considerations-for-achieving-crypto-agility/final>
- [58] NIST FIPS 203, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” 2024. <https://csrc.nist.gov/pubs/fips/203/final>
- [59] NIST FIPS 204, “Module-Lattice-Based Digital Signature Standard,” 2024. <https://csrc.nist.gov/pubs/fips/204/final>
- [60] Kubernetes Documentation, “Service” and “EndpointSlices.” <https://kubernetes.io/docs/concepts/services-networking/service/>
- [61] Istio Documentation, “Virtual Service.” <https://istio.io/latest/docs/reference/config/networking/virtual-service/>
- [62] Open Policy Agent, “OPA for Kubernetes Admission Control.” <https://www.openpolicyagent.org/docs/kubernetes>
- [63] OpenTelemetry, “Traces.” <https://opentelemetry.io/docs/concepts/signals/traces/>
- [64] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, “Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study,” *IEEE Transactions on Software Engineering* 47(2), 243–260, 2021.
- [65] Y. Gan et al., “An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems,” ASPLOS 2019.
- [66] USPTO, “Provisional Application for Patent.” <https://www.uspto.gov/patents/basics/apply/provisional-application>
- [67] WIPO, “How to Protect Inventions through Patents.” <https://www.wipo.int/en/web/patents/protection>
- [68] European Patent Office, Guidelines (PCT), G-IV, 1, 2026. https://www.epo.org/en/legal/guidelines-pct/2026/g_iv_1.html

Appendix A. Reference Specification and Claim Set (Defensive Publication)

Notice. This appendix describes the MIAM mechanism in patent-specification form so that it is stated with claim-level precision and enters the public record. It is published openly as part of independent open-source cybersecurity research. It is not a filed patent application, legal opinion, patentability determination, or freedom-to-operate opinion.

SYSTEMS AND METHODS FOR MISSION-INVARIANT DYNAMIC SERVICE-GRAPH MORPHING WITH PREREQUISITE-ACTIONABILITY REDUCTION, GRAPH-DELTA-DERIVED STATE POLICY, AND EPOCH AUTHORITY ISOLATION

A.1 Technical Field

The disclosure relates to computer and network security, distributed computing, moving-target defense, adaptive software architectures, cloud-native workload orchestration, cryptographic key and identity lifecycle management, and state migration between runtime configurations.

A.2 Background

Distributed applications commonly retain relatively stable service boundaries, dependency relationships, workload identities, execution paths, and state ownership for periods long enough that reconnaissance can be converted into exploit plans. Existing moving-target mechanisms may alter addresses, ports, hosts, containers, execution environments, software variants, or defensive layers. Other systems reconstruct or merge microservices for performance or maintainability. A technical need remains for a transition mechanism in which candidate application graphs are evaluated against machine-readable attack prerequisites and absolute exposure, the selected graph delta determines which state is permitted to cross into the target, and source/target authority remains isolated during cutover so stale authority and runtime state are not implicitly inherited.

A.3 Summary

In one embodiment, a controller maintains a machine-evaluable mission contract, a capability-state map, and a representation of a first runtime service graph. An attacker-observation model contains attack prerequisites associated with the first graph and earlier graphs, each having an actionability predicate or replay test. A candidate generator applies certified transformations to produce candidate second graphs. For each candidate, the controller evaluates continued prerequisite actionability, absolute exposure, and a source-to-target graph delta. The graph delta and capability-state map are used to compile a state-transition policy. A candidate is eligible only if it satisfies mission, exposure, and transition-safety constraints, and selection materially depends on the prerequisite-actionability evaluation. The selected graph is instantiated as a target security epoch having workload authority distinct from the source epoch. During overlap, source principals are denied direct target-epoch authority and mission-domain state crosses only through a state-continuity gate enforcing the compiled state policy and target validators. After safe-point cutover of a stable service interface, source-epoch authority is invalidated or retired.

A.4 Brief Description of the Drawings

Figure 3 illustrates an example MIAM control plane and source/target runtime graphs. Figure 4 illustrates an example epoch transition lifecycle. Figure 5 illustrates service-boundary transformation of capability units

between epochs. Figure 6 illustrates a State Continuity Firewall. Figure 7 illustrates cryptographic epoch isolation and graph-bound key derivation. Figure 2 illustrates candidate generation, scoring, verification, and selection. Figure 1 illustrates retention of a structure-bound prerequisite under recurrence in a finite certified pool.

A.5 Definitions

A *capability unit* is a unit of application behavior that can be assigned, with other capability units, to a deployable service instance. A *service-boundary partition* is an assignment of capability units to service instances. A *mission contract* is a machine-evaluable set of external behavior, policy, safety, data, and operational constraints. An *attack prerequisite* is information, state, access, location, credential, dependency, timing, vulnerability condition, or other precondition useful in executing an attack step. An *epoch-bound prerequisite* is one invalidated at every epoch transition by construction, such as an epoch token, key, or workload identity; a *structure-bound prerequisite* is one whose actionability depends on which graph is active. An *actionability predicate* is a machine-evaluable test or probabilistic model that estimates whether an attack prerequisite remains usable against a candidate graph. *Absolute exposure* is a measure of attack-graph reachability from entry points to protected assets, and of privilege concentration, computed for a candidate graph independently of the attacker-observation model. A *graph delta* identifies source-to-target changes in capability assignment, dependency edges, implementation, identity/policy, placement, or state ownership. A *capability-state map* associates a capability unit with permitted mission-domain state types, validation rules, reconstruction rules, and prohibited state classes. *Mission-domain state* is typed state explicitly required by the mission contract or capability-state map to continue required application behavior; it is not defined merely by being application data. *Runtime/security state* includes process memory, local credentials, source-epoch tokens or keys, executable temporary artifacts, injected hooks, unregistered locks, or other state designated non-transferable by policy. An *epoch* is a bounded interval during which a selected runtime graph and associated security context are active.

A.6 Detailed Description: Control Plane

Referring to Figure 3, system 100 includes stable service interface 110, mission contract registry 120, observability graph builder 130, attacker-knowledge model 140, candidate graph generator 150, actionability, exposure, and leakage scorer 160, verification authority 170, epoch orchestrator 180, identity/key authority 190, state continuity firewall 200, first runtime graph 210, second runtime graph 220, and evidence ledger 230. Components may be centralized, distributed, replicated, or separated across trust domains. The stable service interface can preserve a client-facing logical service name while routing authorized requests to the active runtime graph.

A.7 Detailed Description: Candidate Graphs and Mission Verification

Candidate generator 150 applies one or more certified transformations to a current graph. A transformation may change assignment of capability units to service instances, merge or split service boundaries, substitute an implementation, insert a mediation service, alter a dependency, change placement or substrate, change protocol or routing, change an identity relationship, or combinations thereof. The verification authority 170 evaluates the candidate against mission contract 120. Verification can include formal constraints, policy evaluation, interface compatibility, state-machine invariants, contract tests, property-based tests, load tests, data-consistency checks, attestation, canary execution, or combinations. A candidate failing a hard invariant is not eligible for automatic activation.

A.8 Detailed Description: Attacker-Knowledge Transfer and Exposure

Attacker-knowledge model 140 stores or estimates attack prerequisites associated with a current or prior graph, cumulatively across epochs. Scorer 160 evaluates each prerequisite against a candidate using a machine-evaluable predicate, attack-graph reachability, policy evaluation, vulnerability reachability, exploit-condition evaluation, or reproducible replay. Structure-bound prerequisites may be evaluated against the certified pool as a whole to estimate a long-run recurrence floor. An aggregate risk value may combine impact weight, possession likelihood, and candidate actionability, but the disclosure is not limited to a weighted-sum equation. Scorer 160 separately computes absolute exposure for each candidate under a pinned, versioned exposure-model specification, and rescores the reference graph and all candidates when that specification changes. Candidate selection is constrained so the actionability evaluation materially affects eligibility or ranking rather than being collected only for reporting, and so a candidate whose absolute exposure exceeds a bound relative to a reference graph is ineligible. A selection policy can additionally penalize performance cost, state risk, predictability, externally observable variant fingerprints, or transition detectability.

A.9 Detailed Description: State Continuity

Referring to Figure 6, a transition planner obtains graph delta and capability-state information at 505. Source runtime 510 exports typed mission-domain state through exporter 520. State Continuity Firewall 530, which may be instantiated fresh for each transition, authenticates the export and enforces a state-transition policy compiled from the graph delta, mission contract, and capability-state map. The firewall validates schema, provenance, authorization, replay sequence, integrity, target-specific business invariants, parser/content-safety policy, and state class. Permitted state can be supplied to target rehydrator 540; process memory, source credentials, executable hooks, and other prohibited runtime/security state are denied. Validation failure can quarantine an object, route it to a repair path, or abort the transition when the state is mission-required. Source and target workloads do not gain direct cross-epoch communication merely because an overlap window exists.

A.10 Detailed Description: Cryptographic Epochs

Referring to Figure 7, fresh epoch key sources 610a and 610b generate independent source-epoch and target-epoch secret material. Canonical graph digests 620a and 620b represent the approved graph and policy context of each epoch. Key derivation functions 630a and 630b derive service- or purpose-scoped keys 640a and 640b using domain-separated, unambiguously encoded context. Signed epoch manifest 650 binds graph identity, contract/policy versions, transition metadata, and state-policy identity. Epoch-scoped workload identity 660 and protected channels 670 are authorized through the active/overlap epoch policy so a source identity is not automatically accepted by a target service even while its certificate remains valid. Source key domain 680 transitions through overlap/cutover state 690 to target key domain 700. At 701, after successful cutover, source live key material and credentials can be revoked, disabled, deleted, quarantined, expired, or otherwise made unusable for active authorization. Algorithms may be selected through a versioned cryptographic policy to support crypto agility.

A.11 Detailed Description: Transition and Rollback

Referring to Figure 4, the system observes a current epoch at 300, generates candidates at 310, scores prerequisite actionability and exposure at 320, verifies mission and transition policy at 330, prepares a target epoch at 340, transfers and rehydrates validated state at 350, establishes a transition safe point and cuts

over the stable interface at 360, revokes or retires source-epoch authority at 370, and measures outcomes and updates estimates at 380. Before ownership moves, affected workloads can be fenced from new writes, in-flight operations drained or checkpointed under a registered protocol, and queue/lease state transferred only if declared by the capability-state map. The target can initially receive zero or limited client traffic, followed by weighted canary traffic. If an invariant, consistency condition, security condition, state-policy condition, or SLO threshold is violated, the orchestrator aborts and restores an eligible prior or safe graph. Evidence from the transition is stored for later scoring and calibration.

A.12 Alternative Embodiments

The runtime graph may comprise microservices, serverless functions, virtual machines, unikernels, operating-system processes, data-processing stages, industrial-control services, network functions, AI inference components, storage gateways, or mixed compute units. Morph timing may be periodic, event driven, risk adaptive, pseudo-random, or policy scheduled. Candidate selection may be centralized or federated, scalarized or Pareto-based. A morph can change protocol, insert or remove a policy-enforcement intermediary, change implementation family, change execution substrate, or reassign selected state ownership where a certified migration protocol exists. Non-morphable components can remain pinned while neighboring components morph. Persistent business data can remain under a separate cryptographic domain while only transient transfer objects and workload authority rotate per epoch. The system may operate in one cloud, multiple clouds, edge environments, private data centers, or hybrid environments.

A.13 Reference Claims

1. A computer-implemented method for protecting a distributed application, the method comprising: maintaining a machine-evaluable mission contract and a capability-state map associating capability units with permitted mission-domain state types, validation rules, reconstruction rules, and prohibited runtime or security state classes; operating a first runtime service graph during a first security epoch; obtaining an attacker-observation model comprising a plurality of attack prerequisites associated with the first runtime service graph, respective attack prerequisites being associated with machine-evaluable actionability predicates or replay tests; generating, under a constrained architecture grammar, a plurality of candidate runtime service graphs that differ from the first runtime service graph in at least one capability assignment, dependency, implementation, placement, protocol, or authority relationship; for respective candidate runtime service graphs, evaluating the actionability predicates or replay tests to determine continued actionability of at least a portion of the attack prerequisites; determining a graph delta between the first runtime service graph and a respective candidate runtime service graph; compiling, from at least the graph delta and the capability-state map, a state-transition policy for the respective candidate runtime service graph; rejecting a candidate that fails the mission contract or the state-transition policy; selecting a second runtime service graph based at least in part on a reduction in continued prerequisite actionability among candidates that pass the rejecting; instantiating the second runtime service graph in a second security epoch with second-epoch workload authority distinct from first-epoch workload authority; isolating source-epoch workload authority from direct target-epoch service authorization during an overlap interval except through an authorized transition channel; transferring through a state-continuity gate only mission-domain state permitted by the state-transition policy while preventing transfer of at least one prohibited runtime or security state class; directing a stable logical service interface to the second runtime service graph after satisfaction of a transition-safety condition; and invalidating at least a portion of the first-epoch workload authority after the directing.

2. The method of claim 1, wherein the graph delta includes fusion of at least two capability units into a common service instance, fission of capability units into different service instances, insertion of a mediation boundary, or removal of a direct dependency edge.
3. The method of claim 1, wherein an actionability predicate evaluates at least one of network reachability, authorization of a credential, presence of a vulnerable implementation, existence of a trust edge, exploit-precondition satisfaction, or access to a protected state resource.
4. The method of claim 1, wherein continued prerequisite actionability is estimated from repeated replay outcomes, and an aggregate reconnaissance-transfer value is computed from respective prerequisite actionability estimates, importance weights, and possession-likelihood values representing an estimated probability that an attacker holds the respective prerequisite.
5. The method of claim 1, further comprising computing survival of a multi-step attack path as a joint probability of required prerequisite events, and using a product of marginal actionabilities only when a conditional-independence condition is assumed or validated.
6. The method of claim 1, wherein selecting the second runtime service graph further depends on at least one of absolute exposure, mutation cost, service-level-objective risk, state-continuity risk, architecture distance, configuration predictability, external variant fingerprint leakage, or transition detectability.
7. The method of claim 1, wherein the constrained architecture grammar comprises production rules or prevalidated variation points that preserve one or more mission invariants by construction or are rejected by a verification authority before activation.
8. The method of claim 1, wherein the attacker-observation model is updated from provenance-bearing runtime telemetry, threat detections, red-team evidence, attack-graph analysis, vulnerability reachability, or combinations thereof, and a morph trigger is subject to a rate limit or hysteresis rule.
9. The method of claim 1, wherein the second-epoch workload authority comprises a second-epoch cryptographic master key generated independently of a first-epoch cryptographic master key.
10. The method of claim 9, further comprising generating a canonical digest of the second runtime service graph and deriving a service-scoped or purpose-scoped key using a key derivation function having an unambiguously encoded derivation context that includes an identifier of the second security epoch and the canonical digest.
11. The method of claim 9, wherein a mission-domain-state transfer object is protected with authenticated encryption under source-epoch transfer-key material, validated by the state-continuity gate, and re-protected under target-epoch transfer-key material, the source and target transfer-key material belonging to different epoch authority domains.
12. The method of claim 11, wherein a nonce for authenticated encryption is allocated from a uniqueness-preserving sequence local to an instance of the state-continuity gate whose transfer key is derived with a never-reused instance identifier, and associated authenticated data comprises two or more of a key identifier, a source-epoch identifier, a target-epoch identifier, a transfer direction, a digest of a signed transition manifest committing to source and target graph digests, a service identifier, a state-schema identifier, an object identifier, or a monotonic sequence value.
13. The method of claim 1, wherein compiling the state-transition policy maps a source capability owner to a target capability owner and denies transfer of at least one of process memory, source-epoch private credentials, source-epoch bearer tokens, loaded executable hooks, temporary executable artifacts, unregistered lock state, or local runtime secrets.

14. The method of claim 1, wherein the transition-safety condition comprises at least one of draining an in-flight operation, establishing a write fence or lease epoch, transferring an approved queue offset, reacquiring a distributed lock under a new fencing token, or forcing reconnection of a long-lived authenticated session.
15. A distributed computing system comprising: a mission-contract registry storing a machine-evaluable mission contract and capability-state map; a graph-observation component configured to obtain a first runtime service graph; an attacker-knowledge component storing attack prerequisites associated with machine-evaluable actionability predicates or replay tests; a constrained candidate generator configured to produce candidate service graphs; a scorer configured to evaluate continued actionability of the attack prerequisites for respective candidates; a transition planner configured to determine a source-to-target graph delta and compile a state-transition policy from the graph delta and capability-state map; a verification authority configured to reject a candidate that violates the mission contract or state-transition policy and to authorize an approved transition manifest; an epoch orchestrator configured to instantiate a selected candidate only under an approved transition manifest; an identity or key authority configured to provide the target epoch with workload authority distinct from a source epoch; and a state-continuity firewall configured to transfer permitted mission-domain state according to the compiled state-transition policy while blocking a designated runtime or security state class, wherein the epoch orchestrator is further configured to enforce source-to-target epoch isolation during overlap, move a stable logical service interface after a transition-safety condition, and invalidate source-epoch workload authority after cutover.
16. The system of claim 15, wherein the verification authority is logically or administratively isolated from the candidate generator and signs the approved transition manifest such that the candidate generator cannot unilaterally activate a graph or waive a mission invariant.
17. The system of claim 15, wherein the scorer is configured to estimate external variant fingerprint leakage using cross-validated classification of latency, response size, error behavior, protocol behavior, resource footprint, or other externally observable features associated with candidate service graphs.
18. The system of claim 15, wherein the epoch orchestrator is configured to fence new writes to at least one affected source capability, drain or checkpoint registered in-flight work, perform weighted or canary routing to the target epoch, and roll back to an eligible graph upon violation of an invariant or threshold.
19. The system of claim 15, further comprising an evidence ledger configured to record a graph digest, graph delta, mission-contract version, state-policy version, cryptographic-policy version, verification evidence, actionability-estimate provenance, transition time, and source-to-target epoch relationship.
20. The system of claim 15, wherein the transition planner classifies a workload boundary into at least a directly morphable class and a pinned non-morphable class, and the epoch orchestrator excludes the pinned non-morphable class from automatic service-boundary repartitioning.
21. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause the processors to: maintain a mission contract and capability-state map; generate candidate mission-equivalent service graphs from a first service graph under a constrained grammar; evaluate machine-readable attack-prerequisite actionability against the candidate graphs; determine a graph delta for a candidate; compile from the graph delta and capability-state map a state-transition policy; reject a candidate that violates the mission contract or state-transition policy; select a target graph based at least in part on reduced prerequisite actionability; establish a target security epoch with authority distinct from a source security epoch; enforce cross-epoch authorization isolation during an overlap interval; cause only mission-domain state permitted by the compiled state-transition policy to

cross a state-continuity boundary while blocking a designated source-runtime state class; route a stable service interface to the target graph after a transition-safety condition; and invalidate source-epoch authority after target activation.

22. The non-transitory computer-readable medium of claim 21, wherein the instructions further cause the processors to choose a morph schedule based on an estimated attacker completion-time distribution, an empirically measured per-morph retention factor for epoch-bound prerequisites, a measured long-run recurrence floor for structure-bound prerequisites, and a permitted operational-overhead budget.
23. The non-transitory computer-readable medium of claim 21, wherein cryptographic algorithms are selected by a versioned cryptographic policy such that an algorithm can be replaced without changing the service-graph morphing control logic.
24. The method of claim 1, wherein a transition policy uses, for epoch-bound prerequisites, a knowledge-retention half-time $T_{50,\text{det}} = \tau \ln 2 / \ln(1/\rho)$ for a deterministic morph interval τ and measured per-morph retention factor ρ , or $T_{50,\text{Pois}} = \ln 2 / [\lambda(1 - \rho)]$ for a Poisson morph-event rate λ ; computes, for structure-bound prerequisites, a long-run recurrence floor equal to the stationary probability, under the candidate-selection policy, of graphs in which the prerequisite remains actionable; and rejects a morph schedule or selection policy whose half-time exceeds a policy-defined maximum retention time or whose recurrence floor exceeds a policy-defined maximum.
25. The method of claim 1, wherein selecting the second runtime service graph further requires a normalized service-partition Variation of Information $VI / \log_2 n$ or another structural-diversity measure to exceed a minimum threshold while a mission-defined service-level-objective bound remains satisfied.
26. The system of claim 15, wherein a persistent business-data encryption domain is cryptographically separated from transient target-epoch transfer keys such that retirement of the transient target-epoch keys does not render required persistent business data unrecoverable.
27. The method of claim 1, further comprising computing, for respective candidate runtime service graphs and independently of the attacker-observation model, an absolute exposure from attack-graph reachability between entry points and protected assets and from concentration of sensitive capabilities within a single service instance, and rejecting a candidate whose absolute exposure exceeds that of a fixed reference graph by more than a policy-defined margin, wherein the candidate and the reference graph are scored under the same versioned exposure-model specification and a change of that specification causes the reference graph and all candidates to be rescored and the rebaselining to be recorded.
28. The method of claim 1, wherein the attack prerequisites are classified as epoch-bound prerequisites invalidated at each epoch transition or structure-bound prerequisites whose actionability depends on the active graph, and continued actionability of a structure-bound prerequisite is evaluated against a plurality of graphs in the constrained architecture grammar rather than only against the second runtime service graph.
29. The method of claim 11, wherein the state-continuity gate is instantiated for a single transition from a signed image, receives transfer-key material derived with an instance identifier that a key authority generates, binds into a signed transition manifest, and refuses to reuse, and is destroyed after the transition commits or aborts, and wherein the gate selects a decryption key from the signed transition manifest rather than from a field of the transfer object.
30. The method of claim 1, wherein a mission-required object that fails validation at the state-continuity gate is routed to a quarantine-and-repair path subject to a bounded retry budget, repeated failures attributable to a common object class or origin are raised as a security incident, and a duration during

which transitions are blocked is recorded in an evidence ledger, and wherein, when the duration exceeds a threshold, epoch keys and workload identities of the pinned graph are rotated in place and the capability owning the failing object class is isolated behind a policy-enforcement point.

31. The system of claim 15, wherein the scorer is further configured to estimate transition detectability as a cross-validated detection advantage of a classifier distinguishing observation windows that contain an epoch transition from windows that do not, and the selection penalizes candidates or schedules with higher transition detectability.
32. The system of claim 15, wherein workload identities encode the security epoch and are issued under an epoch-specific intermediate certificate authority whose trust bundle is withdrawn when the source epoch is retired.

A.14 Abstract of the Reference Specification

Systems and methods are disclosed for dynamically changing an internal service architecture of a distributed application while preserving a machine-evaluable mission contract. A controller evaluates machine-readable attack prerequisites and absolute exposure against candidate runtime graphs, determines a source-to-target graph delta, and compiles a state-transition policy from the graph delta and a capability-state map. A verified target graph is instantiated in a distinct security epoch. During overlap, source authority is isolated from target service authorization, and only validated mission-domain state permitted by the compiled transition policy crosses a state-continuity boundary. After a transition-safety condition, a stable logical service interface is cut over and source-epoch authority is invalidated. Candidate selection may additionally account for attack-path survival, recurrence floors, mutation cost, service-level objectives, variant and transition fingerprint leakage, architecture distance, and state risk.

Appendix B. Reference Numerals

Reference numeral	Element
100	MIAM system
110	Stable service interface
120	Mission Contract Registry
130	Observability Graph Builder
140	Attacker-Knowledge Model
150	Candidate Graph Generator
160	Actionability / Exposure / Leakage Scorer
170	Verification Authority
180	Epoch Orchestrator
190	Identity/Key Authority
200	State Continuity Firewall
210	Source runtime graph, epoch e
220	Target runtime graph, epoch $e+1$
230	Evidence Ledger
300–380	Epoch lifecycle steps (Figure 4)
411–415	Capability units A to E
420–460	Example service partitions
470	Service-boundary morph
505–580	Graph-delta state-policy and continuity components
610a/610b–640a/640b	Source-epoch (a) and target-epoch (b) key components
650–701	Transition-boundary cryptographic components
810–890	Candidate selection components

Reference numerals identify elements and are independent of figure numbers.

Appendix C. Disclosure and Availability Note

This work is intentionally public. It is released as independent open-source cybersecurity research so that the mechanisms described here can be reviewed, reproduced, criticized, and improved by the security community, and so that the integrated mechanism is part of the public record as prior art. Under most patent systems, matter made available to the public before a filing date forms part of the prior art [68]; the United States provides a limited one-year grace period for certain inventor-originated disclosures, and pre-filing disclosure may preclude patenting in many other jurisdictions [66, 67]. Readers building on this work should conduct their own freedom-to-operate review with respect to the third-party patents discussed in Section 3.